Computers & Security 173 (2027) 105213

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

AdaGuard: Temporal-leakage-free static–dynamic adaptation for android malware detection under concept drift

Linfeng Yang [a,c,*], Zhi Xie [a], Zhiding Wu [b], Yuan Meng [a], Zhenrong Yang [a]

[a] School of Computer Electronics and Information, and Guangxi Key Laboratory of Multimedia Communication and Network Technology, Guangxi University, Nanning, 530004, China
[b] Datang Hydropower Science and Technology Research Institute Co., Ltd., Chengdu, 610083, China
[c] Key Laboratory of Parallel, Distributed and Intelligent Computing(Guangxi University), Education Department of Guangxi Zhuang Autonomous Region, Nanning, 530004, China

ARTICLE INFO

Dataset link: https://github.com/linfengYang/AdaGuard

Keywords:
Android malware detection
Concept drift
Static and dynamic analysis
Graph-based learning
Temporal adaptation

ABSTRACT

Android malware detectors trained on historical applications often suffer performance degradation on later-year applications because malware behaviors, API usage, and benign application patterns evolve over time. Graph-based detectors can model structural relations within Android applications or between applications and APIs, but their robustness under chronological concept drift remains insufficiently studied, as many existing works still rely on static data splits. To address these challenges, we propose AdaGuard, a temporal-leakage-free static–dynamic adaptation framework for Android malware detection. AdaGuard constructs a temporally isolated static semantic graph from the data available at each deployment stage and introduces a dynamic runtime-evidence channel for uncertain static predictions. A confidence-based late-fusion strategy preserves reliable static decisions and incorporates dynamic evidence only when necessary. During temporal adaptation, AdaGuard updates the detector with historical labeled data and limited labeled target-year samples, while memory-anchored regularization stabilizes the representation space and preserves historical decision consistency. Experiments on a five-year AndroZoo dataset from 2013 to 2017 show that AdaGuard achieves the best average target-year coverage-aware F1 under 5%, 10%, and 15% label budgets. The multi-seed evaluation further shows that AdaGuard with only 5% labeled target-year data achieves an average target-year F1-score of 0.7218, closely matching the 0.7224 obtained by the strongest updated baseline using a 15% labeling budget, demonstrating strong label efficiency.

1. Introduction

The Android operating system has become one of the most widely used mobile platforms, making Android applications a major target for malware attacks. Modern Android malware has evolved from simple privacy leakage and unauthorized billing to more complex behaviors, such as payload hiding, dynamic loading, API obfuscation, and attacks against emerging mobile software ecosystems (Huang et al., 2024). To identify these threats, data-driven malware detection methods have been widely studied. By learning malware-related patterns from static program structures or dynamic execution traces, these methods can achieve strong detection performance when training and testing samples follow similar distributions (Muzaffar et al., 2022).

However, real-world deployment over time rarely satisfies such a stable data distribution. Android applications, system APIs, third-party libraries, and malware behaviors change continuously over time. As a result, a detector trained on historical applications may face a shifted distribution when applied to applications collected in later years. This temporal distribution shift, commonly referred to as concept drift, can significantly reduce the reliability of malware detectors. Prior longitudinal studies have shown that malware classifiers may suffer clear performance degradation when evaluated in chronological settings rather than on randomly mixed datasets (Chen et al., 2023; Haque et al., 2026; Sabbah et al., 2025). Therefore, robust Android malware detection requires not only effective feature learning, but also temporally sound evaluation and adaptation mechanisms under evolving data distributions (Sun et al., 2025).

Existing Android malware detection methods still face three important challenges in chronological deployment. First, graph-based methods have shown strong ability in modeling structural relations within Android applications or among applications, APIs, and code entities

* Corresponding author.
E-mail addresses: ylf@gxu.edu.cn (L. Yang), xiezhi0314@163.com (Z. Xie), zdedward@163.com (Z. Wu), yuanm1073@163.com (Y. Meng), zhenrongy111@163.com (Z. Yang).

https://doi.org/10.1016/j.cose.2026.105213
Received 19 July 2026; Received in revised form 28 September 2026; Accepted 14 October 2026
Available online 21 October 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(Shen et al., 2024; He et al., 2023). However, many of them are still evaluated under static data splits, where temporal order is not the main concern. In chronological deployment, global graph construction must be staged carefully, because introducing unseen target-year test APKs before model updating or validation may affect graph topology, graph statistics, and message passing even without using their labels. This implicit information flow can lead to overly optimistic estimates of time-aware generalization. Therefore, chronological graph-based malware detection requires training and updating graphs to be constructed only from the data available at each deployment stage.

Second, single-modality detectors are often insufficient for long-term robustness. Static analysis is efficient and scalable, but it can be affected by API evolution, code obfuscation, and feature aging (Zhang et al., 2020). Dynamic analysis provides useful runtime evidence, but it requires executing applications in a sandbox and may suffer from sandbox evasion, incomplete execution coverage, missing reports, and false positives caused by benign runtime behaviors (Onwuzurike et al., 2019). These limitations suggest that static structural evidence and dynamic runtime evidence should be used in a complementary and cost-aware manner. In particular, dynamic evidence should be incorporated mainly when static predictions become uncertain under distribution shift, rather than being forced to participate in every prediction.

Third, practical temporal adaptation is constrained by limited labels from the target year. In real deployment, labels for newly collected applications are expensive because they often rely on expert inspection or delayed antivirus reports. Updating a detector with a small number of target-year samples can improve adaptation to new distributions, but it may also make the representation space and decision boundary unstable, especially when the newly labeled data are limited or imbalanced. Therefore, temporal adaptation should jointly use accumulated historical labeled memory and limited target-year labels, while stabilizing the representation space and preserving historical decision consistency during year-by-year updating.

To address these challenges, we propose AdaGuard, a temporal-leakage-free static–dynamic adaptation framework for Android malware detection under concept drift. AdaGuard consists of a static semantic graph channel and a dynamic runtime-evidence channel. In the static channel, AdaGuard constructs a temporally isolated APK–API-cluster semantic graph from the data available at each deployment stage. Instead of building a raw APK–API graph with a large number of individual API nodes, AdaGuard represents semantically related APIs as API-cluster nodes and connects APKs to the API clusters they invoke. The graph further uses fixed API-cluster mappings, source-only cluster-IDF edge weights, and semantic links between API clusters. This compact graph design reduces graph scale, alleviates sensitivity to API-level changes, and supports leakage-free temporal updating. A learnable feature masking mechanism is further used to reduce drift-sensitive structural noise before graph-based message passing.

In the dynamic channel, AdaGuard treats runtime behaviors as complementary evidence for statically uncertain samples. Since dynamic analysis usually requires sandbox execution and may incur substantial runtime cost, AdaGuard does not force dynamic evidence to dominate all predictions. Instead, it adopts an uncertainty-triggered fusion design: confident static predictions are preserved, while calibrated dynamic runtime evidence is incorporated only for ambiguous cases. The two channels are integrated through a confidence-based late-fusion strategy, which maintains the efficiency and stability of static analysis while allowing runtime behaviors to assist difficult samples.

For temporal adaptation, AdaGuard updates the model using accumulated historical labeled memory and limited labeled samples from each target year. To stabilize this process, AdaGuard introduces memory-anchored regularization, which combines elastic prototype alignment, source-aware knowledge distillation, and gradient-reversal-based domain alignment. Rather than relying only on newly labeled target samples, this mechanism stabilizes the representation space, preserves historical decision consistency, and reduces year-specific representation bias during temporal updating. This design is suitable for budget-constrained malware analysis, where only a small fraction of target-year applications can be labeled for model updating.

The main contributions of this paper are summarized as follows:

- We propose AdaGuard, a temporal-leakage-free static–dynamic adaptation framework for Android malware detection under chronological concept drift. AdaGuard updates the detector year by year using accumulated historical labeled memory and limited labeled target-year samples, thereby improving temporal robustness under budget-constrained deployment.
- We design a compact static–dynamic evidence learning scheme within AdaGuard. In the static channel, AdaGuard represents semantically related APIs as API-cluster nodes and builds a temporally isolated APK–API-cluster semantic graph for chronological detection, which reduces graph scale, alleviates sensitivity to API-level changes, and supports leakage-free temporal updating. In the dynamic channel, runtime evidence is incorporated only for statically uncertain samples through confidence-based late fusion, allowing dynamic behaviors to assist difficult cases while reducing unnecessary reliance on costly sandbox analysis.
- We introduce a memory-anchored temporal adaptation strategy for accumulated-memory updating with limited target-year labels. By combining elastic prototype alignment, source-aware knowledge distillation, and gradient-reversal-based domain alignment, AdaGuard stabilizes the representation space, preserves historical decision consistency, and reduces year-specific representation bias during temporal updating.
- We evaluate AdaGuard on a five-year AndroZoo dataset from 2013 to 2017 under 5%, 10%, and 15% target-year label budgets. The results show that AdaGuard achieves stronger temporal robustness and label-budget efficiency than the compared baselines. Notably, the multi-seed evaluation shows that AdaGuard using only 5% labeled target-year data achieves an average target-year F1-score very close to that of the strongest baseline using a 15% labeling budget, indicating strong label efficiency.

2. Related work

This section reviews prior studies related to AdaGuard from three aspects: static, dynamic, and hybrid detection methods; graph-based detection and API semantic abstraction; and concept drift adaptation with limited target-year labels.

2.1. Static, dynamic, and hybrid detection methods

Early data-driven Android malware detectors mainly rely on static features extracted from application manifests and bytecode. Drebin (Arp et al., 2014) is a representative static detector that performs broad static analysis over the manifest and disassembled dex code. It extracts multiple feature sets, including hardware components, requested permissions, application components, filtered intents, restricted API calls, used permissions, suspicious API calls, and network addresses, and maps them into a high-dimensional sparse feature space for linear classification. Static methods are generally efficient, and Drebin further provides explainable detection results by tracing important features that contribute to each decision. However, the distributions of static features may change as Android APIs, third-party libraries, benign applications, and malware behaviors evolve over time.

To improve robustness against API-level changes, API-call abstraction and behavioral modeling methods have been proposed. MaMaDroid (Onwuzurike et al., 2019) abstracts API calls into higher-level semantic groups, such as API families, packages, or classes, and models the resulting abstracted call sequences from static call graphs as Markov chains. This abstraction reduces the dependence on raw

API names and improves resilience to Android API evolution. These methods show the value of API-level abstraction for long-term malware detection, but they mainly rely on static behavioral representations and are not designed for budget-constrained chronological adaptation with limited target-year labels.

Dynamic analysis provides another perspective by executing applications in controlled environments and collecting runtime behaviors. Runtime evidence can complement static representations by exposing behaviors that are difficult to observe from application code alone. Nevertheless, dynamic analysis usually requires sandbox execution and may suffer from sandbox evasion, incomplete execution coverage, missing reports, and false positives caused by benign runtime behaviors.

Hybrid and multi-view detectors exploit complementary evidence from different feature spaces and analysis channels. Chimera (Oliveira and Sassi, 2020) jointly models manifest features, DEX images, and system-call sequences through dedicated subnetworks and intermediate fusion, while DroidDetectMW (Taher et al., 2023) applies separate feature-selection procedures to static and dynamic attributes for malware detection and family classification. Other multi-view methods combine complementary static representations. DeepCatra (Wu et al., 2023) integrates BiLSTM-based opcode-sequence modeling with GNN-based abstract flow-graph learning, whereas SAC (Yang et al., 2025) jointly captures local content features from DEX visualization and global structural features from bytecode relationship graphs. Wang et al. (2022), by contrast, adopt a staged pipeline in which permission-sequence analysis first identifies suspicious applications, after which runtime object-reference graphs are examined for further classification. Collectively, these studies demonstrate the effectiveness of multimodal fusion, multi-view representation learning, and staged analysis. However, they either fuse multiple views uniformly or follow a predefined processing pipeline, rather than using calibrated static uncertainty to determine, on a per-sample basis, whether dynamic runtime evidence is required. AdaGuard further integrates this uncertainty-triggered mechanism with chronological adaptation under concept drift.

2.2. Graph-based detection and API semantic abstraction

Graph-based Android malware detection has attracted increasing attention because graph representations can preserve relational and topological information that is difficult to retain in flat feature vectors. Application-level approaches typically construct a separate graph or subgraph for each APK. DGCNDroid (Yang et al., 2021) extracts function-call subgraphs containing sensitive APIs and applies a deep graph convolutional network to capture node attributes and multi-hop topological information. MsDroid (He et al., 2023) identifies malicious snippets by constructing behavior subgraphs around permission-related APIs and performing snippet-level graph learning. Yumlembam et al. (2023) generate GNN-based embeddings from application-specific API graphs and combine them with permission and intent features for Android malware detection. These methods demonstrate the effectiveness of intra-application graph modeling, but each APK is represented largely in isolation, limiting the direct exploitation of semantic and structural relations shared across applications.

To capture such cross-application dependencies, heterogeneous graph methods model richer relations among applications and security-related entities. HAWK (Hei et al., 2024) constructs a heterogeneous information network with applications and multiple security-related entities, such as permissions, APIs, classes, interfaces, and native libraries, and uses meta-structure-guided graph attention to learn application representations. GHGDroid (Shen et al., 2024) constructs a global heterogeneous graph with APK nodes and sensitive API nodes, where APK–SAPI and SAPI–SAPI edges are used to learn APK embeddings for detection. By modeling cross-application relations and API-level dependencies, global graph methods can better capture shared semantic and structural patterns across applications.

However, the generalization of these representations depends on whether the node attributes and connectivity patterns captured in the training graph remain informative at deployment. As new APKs emerge and API usage patterns evolve, shifts in both node features and graph structure may degrade performance on later-year applications (Wu et al., 2024). Existing global graph-based detectors are mostly evaluated under static or non-chronological data splits, where temporal order is not the main concern. However, when these detectors are used in chronological deployment, graph construction itself becomes part of the temporal protocol. If unseen target-year test APKs are introduced before model updating or validation, they may influence graph topology, graph statistics, and message passing even without using their labels. This implicit information flow can lead to overly optimistic estimates of time-aware generalization. Therefore, graph-based Android malware detection under concept drift requires temporally isolated graph construction, where training and updating graphs are built only from data available at the current deployment stage.

In addition to graph construction, API semantic abstraction is also important for temporal robustness. APIGraph (Zhang et al., 2020) constructs an API relation graph from Android API documentation and extracts semantic API clusters to enhance existing classifiers against evolved malware. SDAC (Xu et al., 2022) learns API vectors from API-call contexts and clusters APIs according to semantic distances, so that newly appeared APIs can be associated with existing semantic features. These studies show that API semantics can reduce the dependence on raw API names and help mitigate model aging caused by malware evolution and API changes. Different from API-level semantic abstraction methods that mainly use API clusters as enhanced features, AdaGuard uses semantic API clusters as graph nodes and builds a compact APK–API-cluster semantic graph for chronological detection and adaptation. This design reduces the graph scale compared with raw APK–API graphs, while enabling leakage-free graph learning under year-by-year temporal updating.

2.3. Concept drift adaptation with limited target-year labels

Android malware detection is naturally affected by concept drift because malware families, evasion strategies, API usage patterns, and benign application distributions change over time.

A fundamental concern in this setting is whether evaluation protocols realistically preserve temporal order. TESSERACT (Pendlebury et al., 2019) systematically showed that temporally inconsistent train–test configurations can lead to overly optimistic malware-classification results and proposed spatio-temporal constraints for more realistic malware evaluation across time.

Prior studies have modeled malware detection as an evolving data-stream problem (Ceschin et al., 2023). Recent longitudinal benchmarks such as LAMDA (Haque et al., 2026) further show that detectors can perform well under in-distribution evaluation but degrade when tested on temporally distant applications. These findings highlight the importance of temporally realistic and chronological evaluation for Android malware detectors.

Several methods have been proposed to mitigate model aging under concept drift. TRANSCENDENT (Barbero et al., 2022) uses conformal evaluation to assess prediction quality and rejects predictions on samples that are likely to be misclassified under distribution shift. This rejection-based design can improve the reliability of accepted predictions, but it also reduces effective decision coverage because some samples are quarantined rather than directly classified. Therefore, when comparing rejection-based methods with full-coverage detectors, coverage-aware metrics are needed to reflect the trade-off between reliability and coverage.

Other studies mitigate concept drift by updating malware models with newly collected samples. Continuous learning for Android malware detection (Chen et al., 2023) uses active learning to select informative samples for analyst labeling, adds the labeled samples to

the training set, and retrains the classifier over time. LDCDroid (Liu et al., 2025) learns data drift characteristics to guide the use of pseudo-labeled and true-labeled samples for handling model aging. TIML (Sun et al., 2025) studies temporal-incremental learning for Android malware family classification, where malware families may persist across multiple time periods and exhibit distribution shifts over time. More recent studies further explore reinforcement-learning-based classification, active learning, and rejection policies (McFadden et al., 2026), and explanation-guided drift detection and adaptation (He et al., 2025). These works show that temporal updating is important, but they mainly address different aspects of the problem, such as sample selection, pseudo-label usage, family-level classification, rejection-aware decision making, or explanation-guided updating.

Building on prior work emphasizing temporally realistic evaluation, AdaGuard addresses a related but different problem: full-coverage Android malware detection and adaptation under chronological concept drift with limited target-year labels. Under a controlled evaluation protocol, all updated methods use the same randomly selected labeled target-year samples, which separates the effect of temporal adaptation from the effect of sample-selection strategies. AdaGuard tackles this setting through temporal-leakage-free static–dynamic adaptation. The static channel builds a temporally isolated APK–API-cluster semantic graph for leakage-free chronological updating, the dynamic channel provides uncertainty-triggered runtime evidence for statically ambiguous samples, and memory-anchored regularization stabilizes accumulated-memory updating under limited target-year labels.

3. AdaGuard

AdaGuard follows a year-by-year temporal protocol for Android malware detection under concept drift. As shown in Fig. 1, the framework consists of an initial training phase and a target-year updating phase. Given source-year data, AdaGuard first trains the initial static and dynamic experts, calibrates the fusion parameters, and initializes the source memory under a temporally isolated setting. For each target year, the experts are then updated using accumulated historical labeled data and a small labeled subset from the current year, while the fixed target-test set remains unseen until final inference and evaluation.

AdaGuard integrates two complementary evidence channels. The static semantic graph channel constructs a temporally isolated APK–API-cluster semantic graph to encode APK–API-cluster usage and semantic relations among API clusters. The dynamic runtime-evidence channel provides uncertainty-triggered runtime evidence for statically ambiguous samples. A confidence-based late-fusion strategy preserves reliable static predictions and incorporates dynamic evidence only when needed, thereby avoiding unnecessary reliance on costly sandbox analysis. During target-year updating, memory-anchored temporal adaptation updates the static and dynamic experts using accumulated historical labeled memory and limited labeled target-year samples, producing an updated AdaGuard state for the next year.

The remainder of this section is organized as follows. Section 3.1 defines the temporal detection protocol. Section 3.2 introduces the static and dynamic feature views. Sections 3.3 and 3.4 describe the temporally isolated static semantic graph and its graph encoder. Section 3.5 presents the dynamic runtime-evidence encoder. Section 3.6 introduces the confidence-based late-fusion strategy. Finally, Section 3.7 describes the memory-anchored temporal adaptation process.

3.1. Problem formulation and temporal protocol

Android malware detection under chronological deployment faces concept drift. Let $P_0(X,Y)$ and $P_t(X,Y)$ denote the joint distributions of features X and labels Y in the source year and target year t, respectively. Since Android APIs, benign application patterns, and malware behaviors evolve over time, the source-year and target-year distributions generally satisfy

$$P_0(X,Y) \neq P_t(X,Y), \tag{1}$$

This distribution mismatch characterizes the chronological concept drift considered in this study.

Let D_0 denote the applications collected in the source year. The source-year data are split into a training set D_0^{tr} and a validation set D_0^{val}. The training set is used to train the initial static and dynamic experts, while the validation set is used for model selection and fusion calibration.

For each target year $t = 1,\dots,T$, the target-year applications are divided into a candidate pool C_t and a fixed test set D_t^{test}. Only samples from C_t can be selected and labeled for model updating, while D_t^{test} is reserved for final inference and evaluation. Under a target-year label budget ρ, only a small subset of the candidate pool is labeled:

$$L_t^{\rho} \subset C_t, \qquad |L_t^{\rho}| = \lfloor \rho |C_t| \rfloor, \tag{2}$$

Here, L_t^{ρ} denotes the selected labeled target-year subset, ρ is the label-budget ratio, and $|\cdot|$ denotes the cardinality of a set. The selected labeled subset is further split into a target-year training part L_t^{tr} and a validation part L_t^{val}, where

$$L_t^{\rho} = L_t^{\mathrm{tr}} \cup L_t^{\mathrm{val}}, \qquad L_t^{\mathrm{tr}} \cap L_t^{\mathrm{val}} = \emptyset. \tag{3}$$

The training part is used for expert updating, while the validation part is used for model selection and early stopping. At target year t, AdaGuard updates its static and dynamic experts using the labeled training data available up to the current year. We denote the training memory after incorporating the selected labeled samples from year t as

$$H_t = D_0^{\mathrm{tr}} \cup L_1^{\mathrm{tr}} \cup L_2^{\mathrm{tr}} \cup \cdots \cup L_t^{\mathrm{tr}}. \tag{4}$$

For model selection, the corresponding validation memory is defined as

$$V_t = D_0^{\mathrm{val}} \cup L_1^{\mathrm{val}} \cup L_2^{\mathrm{val}} \cup \cdots \cup L_t^{\mathrm{val}}. \tag{5}$$

Thus, H_t provides the labeled training data used for temporal adaptation, while V_t is used only for validation and model selection. The fixed test sets are excluded from all training, validation, and model-selection steps. The goal of AdaGuard is to update the detector under this temporal protocol and predict every sample in each fixed test set D_t^{test}. In this study, temporal isolation refers specifically to chronological isolation across deployment years and to preventing fixed-test labels from influencing training, validation, model selection, or graph-statistic estimation. It does not imply sample-wise independence among test APKs within the same target year. During final static-graph inference, the test APKs from the same target year are added to the inference graph together, whereas APKs from future years remain excluded.

3.2. Multi-view feature representation

AdaGuard uses two complementary feature views for each APK: static features and dynamic runtime tokens. The static view is extracted from the disassembled application without executing the APK and is used to construct the APK–API-cluster semantic graph. The dynamic view is extracted from dynamic analysis reports and provides runtime behavioral evidence for later static–dynamic fusion.

3.2.1. Static feature representation

For each APK, AdaGuard extracts static features without executing the application. These features include API-cluster indicators, permission features, and opcode statistics. The API-cluster indicators describe which semantic API clusters are invoked by the APK. Raw API invocations extracted from the disassembled code are first normalized into a unified API format and then mapped to semantic API clusters. Let N_c

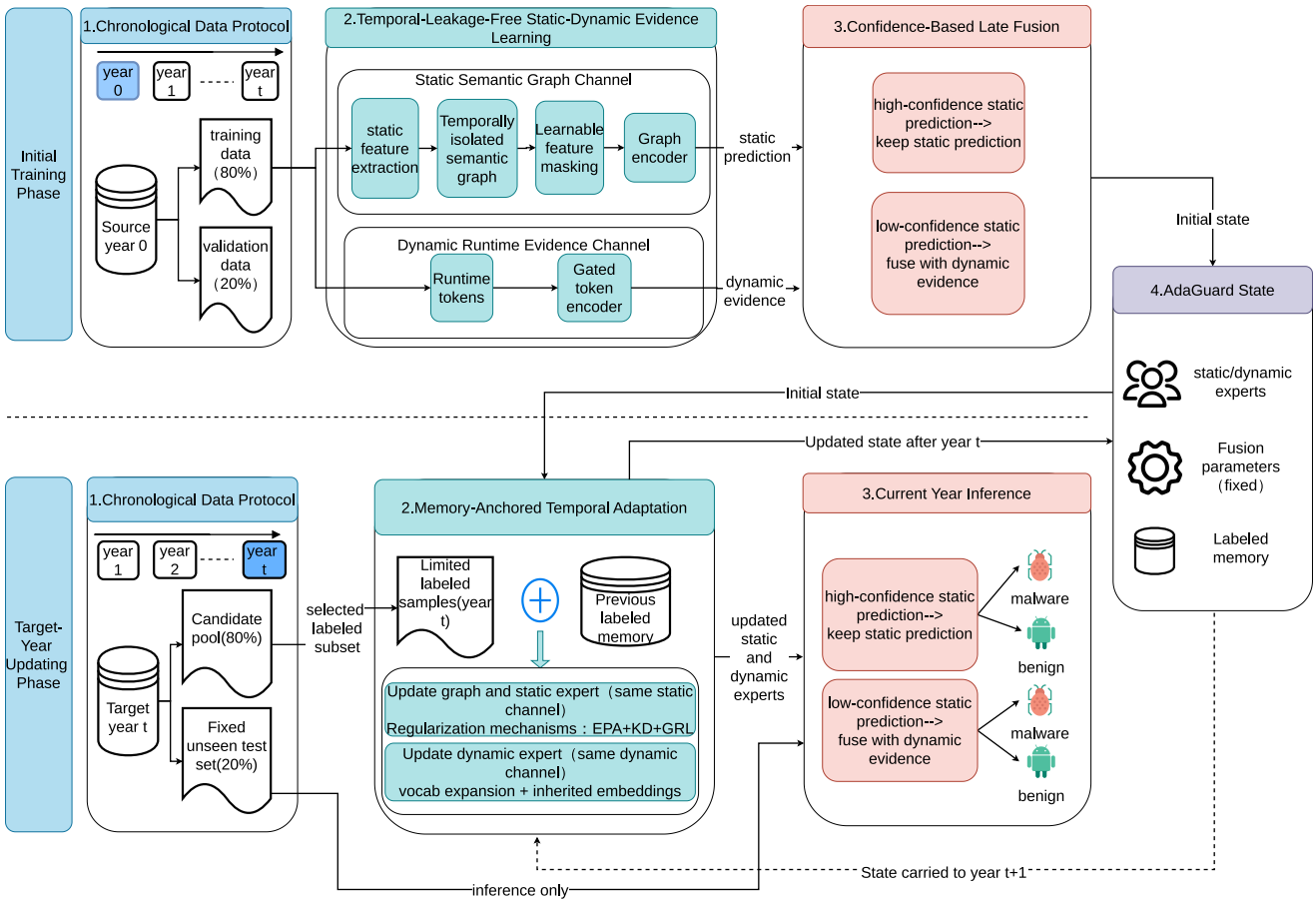


Fig. 1. Overall framework of AdaGuard.

denote the number of API clusters. The API-cluster feature vector of APK u is defined as

$$x_u^c = [x_{u,1}^c, x_{u,2}^c, \ldots, x_{u,N_c}^c], \tag{6}$$

where $x_{u,i}^c = 1$ if APK u invokes at least one API assigned to cluster i, and $x_{u,i}^c = 0$ otherwise. In our implementation, $N_c = 2000$. In addition to API-cluster indicators, AdaGuard uses permission features and opcode statistics to retain complementary static information. The permission feature vector is denoted as $x_u^p \in \mathbb{R}^{D_p}$, where D_p is the number of permission dimensions. The opcode-statistics vector is denoted as $x_u^o \in \mathbb{R}^{D_o}$, where D_o is the number of opcode dimensions. The static feature vector of APK u is obtained by concatenating these three feature groups:

$$x_u^{stat} = [x_u^c : x_u^p : x_u^o] \in \mathbb{R}^{D_{stat}}, \tag{7}$$

where $D_{\text{stat}} = N_c + D_p + D_o$. This vector is used as the initial feature of APK node u in the APK–API-cluster semantic graph. For API-cluster nodes, AdaGuard uses one-hot cluster-identity features and zero-pads the remaining dimensions so that both node types share the same D_{stat}-dimensional input feature space. All node features are further ℓ_2-normalized before feature masking and graph encoding. The construction of graph edges and edge weights is introduced in Section 3.3.

3.2.2. Dynamic runtime token representation

In addition to static features, AdaGuard extracts runtime tokens from dynamic analysis reports. These tokens describe behaviors observed during execution, such as API calls, checked permissions, hooked signals, cryptographic operations, network activities, native library loading, command execution, invoked methods, and system-property lookups. If no valid dynamic report is available, the APK is assigned a special no-data token.

AdaGuard does not treat runtime tokens as a strict execution sequence. Dynamic reports may group behaviors by event fields rather than record a precise temporal order. Therefore, the dynamic view uses runtime tokens as behavioral evidence and aggregates them through a gated token encoder. This design allows the dynamic channel to focus on useful runtime behaviors without relying on a possibly noisy event order.

Before being fed into the dynamic encoder, composite tokens are converted into unigram tokens when applicable. Each token is then mapped to an integer ID according to the current dynamic vocabulary. For mini-batch training, each token list is padded or truncated to a fixed maximum length. In our implementation, the maximum length is 150. Padding positions are filled with the padding token, and tokens outside the current vocabulary are mapped to a shared unknown token.

To maintain temporal isolation, the initial dynamic vocabulary is built only from source-year training samples. During target-year updating, the vocabulary is expanded only with currently available labeled training samples. Runtime tokens from fixed target-test samples are not used to update the vocabulary; unseen tokens in these samples are mapped to the unknown token during inference. The dynamic runtime encoder is introduced in Section 3.5.

3.3. Temporally isolated static semantic graph construction

AdaGuard constructs a weighted APK–API-cluster semantic graph to model relations between applications and semantic API clusters. The graph contains two types of nodes: APK nodes and API-cluster nodes. It also contains two types of edges: APK-to-cluster invocation edges and cluster-to-cluster semantic edges. The invocation edges connect an APK with the API clusters it uses, while the semantic edges connect API clusters with similar semantic roles.

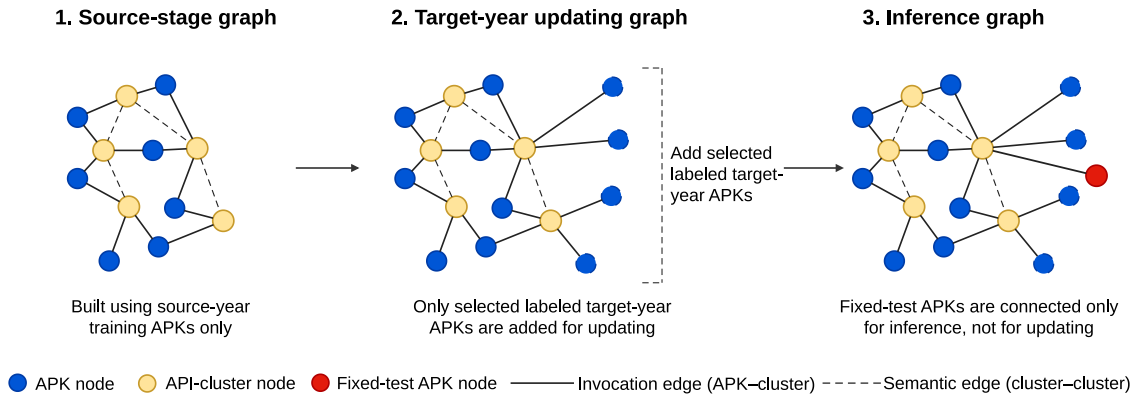


Fig. 2. Temporally isolated construction and usage of the static semantic graph in AdaGuard.

As shown in Fig. 2, AdaGuard uses this graph under a temporal isolation protocol. In the source stage, the training graph is built from source-year training APKs and API-cluster nodes. During target-year updating, only selected labeled target-year APKs are added to the updating graph. Fixed target-test APKs are attached only in the inference graph after graph statistics and model parameters have been fixed, so they do not participate in the construction of training and updating graphs, model updating, validation, or threshold selection. This design allows AdaGuard to exploit cross-application graph relations while avoiding temporal data leakage. The following subsections first describe the initialization of the API-cluster semantic space, followed by the construction of the two edge types and their phase-specific usage.

3.3.1. API-cluster initialization

The API-cluster space used by AdaGuard is initialized from the public semantic resources released with APIGraph (Zhang et al., 2020). APIGraph first extracts API entities from parsed Android API documentation and derives inter-entity relations according to predefined relation templates. These relations are organized into an API relation graph, which is subsequently embedded using TransE (Bordes et al., 2013) so that APIs with similar relational semantics are represented in a common embedding space. The resulting API embeddings are grouped using k-means clustering. Following the released Android configuration of APIGraph, AdaGuard uses $N_c = 2000$ API clusters and the corresponding API-to-cluster mapping. Therefore, API membership is determined by clustering in the TransE semantic embedding space rather than by API occurrence statistics in our chronological APK dataset. For reproducibility, the corresponding API-cluster initialization and resource-loading procedures are provided in our released code. The API-to-cluster mapping and TransE embeddings are fixed throughout the chronological evaluation and are not relearned from target-year APKs.

3.3.2. APK-to-cluster invocation edges

An APK-to-cluster edge is added when an APK invokes at least one API belonging to the corresponding API cluster. If $x^c_{u,i} = 1$, AdaGuard connects APK node u with API-cluster node i.

To maintain temporal isolation, AdaGuard computes the inverse document frequency (IDF) (Sparck Jones, 1972) of each API cluster using only the source-year training set. For the ith API cluster, its source-only IDF is defined as

$$I_i = \log \frac{|D_0^{\mathrm{tr}}|}{n_i + 1}, \tag{8}$$

where I_i denotes the IDF value of the ith API cluster, $|D_0^{\mathrm{tr}}|$ is the number of source-year training applications, and n_i is the number of source-year training applications that invoke the ith API cluster. The $n_i + 1$ avoids division by zero. For clusters with positive IDF values, the raw IDF values are smoothed to obtain the APK-to-cluster edge weight:

$$w_{ac}(u, i) = \alpha + (1 - \alpha) \cdot \frac{I_i - I_{\min}}{I_{\max} - I_{\min} + \epsilon}, \tag{9}$$

where $w_{ac}(u, i)$ denotes the weight of the edge between APK u and the ith API cluster, $I_{\min}$ and $I_{\max}$ are the minimum and maximum positive IDF values computed from the source-year training set, ϵ is a small constant for numerical stability, and α is the minimum positive edge weight. In our implementation, $\alpha = 0.2$. If APK u does not invoke the ith API cluster or $I_i \le 0$, no APK-to-cluster edge is added.

The smoothing parameter α preserves a basic connection strength for valid APK-to-cluster relations, so common but still useful API-cluster relations are not completely suppressed. Meanwhile, clusters with higher source-only IDF values receive larger weights. This design emphasizes more discriminative API-cluster relations while preventing graph sampling and attention aggregation from being dominated by a few extremely large IDF values.

3.3.3. Cluster-to-cluster semantic edges

APK-to-cluster edges describe which API clusters are used by an APK, but they do not directly model semantic relations among API clusters. To capture such relations, AdaGuard further adds semantic edges between API-cluster nodes. Using the fixed TransE API embeddings and API-to-cluster mapping introduced above, AdaGuard constructs a cluster-level semantic backbone from the semantic proximity between API clusters.

Let A_i denote the set of APIs assigned to the ith API cluster, and let e_a denote the TransE embedding of API a. The semantic centroid of the ith API cluster is computed as

$$z_i = \frac{1}{|A_i|} \sum_{a \in A_i} e_a, \tag{10}$$

Here, z_i represents the semantic embedding of the ith API cluster. If a cluster has no valid API embedding, its centroid is treated as a zero vector and it does not form semantic edges. The semantic similarity between the (i)th and (j)th API clusters is measured using cosine similarity, a standard vector-space similarity measure (Salton et al., 1975):

$$s_{ij} = \frac{z_i^\top z_j}{|z_i|_2 |z_j|_2 + \epsilon}, \tag{11}$$

where s_{ij} denotes the semantic similarity between the two API clusters, and ϵ is a small constant for numerical stability. A cluster-to-cluster semantic edge is added when $s_{ij} \ge \tau_c$ and $i \ne j$. The edge weight is then defined as

$$w_{ij} = s_{ij}, \tag{12}$$

In our implementation, $\tau_c = 0.85$. If $s_{ij} < \tau_c$, no semantic edge is added between the two API clusters.

These semantic edges allow information to propagate between API clusters with similar roles. This helps the static graph channel capture relations between semantically related APIs, even when malware variants replace one API with another similar API.

3.3.4. Phase-specific graph usage

The API-to-cluster mapping and the cluster-to-cluster semantic backbone are fixed before target-year updating and remain unchanged throughout the chronological evaluation. Since the API semantic space is initialized independently of target-year APK statistics, AdaGuard enforces temporal isolation by controlling which APK nodes are allowed to connect to this fixed semantic backbone at each stage.

In the source stage, the training graph is built from source-year training APKs and the fixed API-cluster backbone. Source-year validation samples are used for model selection and fusion calibration, but they do not update graph statistics or model parameters. At target year t, the updating graph connects the APKs in the labeled training memory H_t to the same fixed API-cluster backbone. The APK-to-cluster edge weights are computed using the source-only IDF values defined above and are not recomputed from fixed target-test APKs.

During inference, fixed target-test APKs are incorporated into the inference graph on a year-by-year basis. For each target year, the test APKs from that year are added to the corresponding inference graph after graph statistics and model parameters have been fixed. Predictions for these test APKs are then generated in mini-batches using multi-hop neighbor sampling. Since the static encoder performs multi-hop GraphSAGE neighborhood aggregation, the representation of one test APK may indirectly incorporate unlabeled feature information from another test APK through shared API-cluster nodes. Therefore, the static graph channel follows a year-wise transductive inference protocol rather than strictly independent per-APK inference. Their labels are never used, and they do not update graph statistics, edge weights, model parameters, validation decisions, or fusion parameters.

From a computational perspective, the current APK–cluster graph construction identifies the API clusters associated with each APK according to the fixed API-to-cluster mapping. Therefore, constructing APK nodes and their associated APK–cluster edges in the updating graph at target year t requires $O(|H_t|N_c)$ time, where $|H_t|$ denotes the number of historical APKs. Since $N_c = 2000$ is fixed, this cost grows linearly with the number of APKs included in H_t. The construction of the cluster semantic backbone requires $O(N_c^2)$ pairwise similarity evaluations among API-cluster centroids; with the TransE embedding dimension fixed, this also corresponds to $O(N_c^2)$ time. This cost is incurred only once because the API-cluster mapping and the semantic backbone remain unchanged during target-year updating. Moreover, since H_t extends the previous training memory with newly available samples in L_t^{tr}, the graph can be incrementally updated by adding only these new APK nodes and their associated APK–cluster edges to the existing semantic backbone, requiring $O(|L_t^{tr}|N_c)$ time rather than reconstructing all historical APK nodes and associated edges from scratch. The empirical construction time, memory consumption, and incremental-update cost are evaluated in Section 4.2.5.

3.4. Static graph encoder with learnable feature masking

After constructing the temporally isolated APK–API-cluster semantic graph, AdaGuard learns static APK representations with a graph encoder. The encoder first applies a learnable feature mask to the input node features. This mask down-weights redundant or potentially drift-sensitive static dimensions before graph message passing. The masked features are then fed into an attention-based GraphSAGE encoder, which aggregates information from APK-to-cluster invocation edges and cluster-to-cluster semantic edges. The resulting APK representations are used to produce static predictions for later confidence-based fusion.

3.4.1. Learnable feature masking

Static features may contain redundant or unstable dimensions because they combine API-cluster indicators, permission features, and opcode statistics. Directly using all dimensions for graph message passing may propagate noisy feature signals through the graph and affect the representations of neighboring nodes. To reduce this effect, AdaGuard applies a learnable feature mask before graph aggregation.

For each node v in the APK–API-cluster semantic graph, let x_v denote its input feature vector. AdaGuard uses a small neural network $f_m(\cdot)$ followed by a sigmoid function to generate a node-adaptive soft feature mask:

$$m_v = \sigma(f_m(x_v)), \tag{13}$$

where m_v has the same dimension as x_v, and each element of m_v lies between 0 and 1. A larger mask value indicates that the corresponding feature dimension is more strongly preserved, while a smaller value reduces its influence. The masked feature vector is computed by element-wise multiplication:

$$\hat{x}_v = x_v \odot m_v, \tag{14}$$

where $\odot$ denotes element-wise multiplication. The masked feature $\hat{x}_v$ is used as the input to the subsequent graph encoder. To encourage compact feature usage, AdaGuard adds an L_1 regularization term on the learned masks:

$$\mathcal{L}_m = \frac{1}{|B|}\sum_{v\in B}\|m_v\|_1, \tag{15}$$

where B is a mini-batch of sampled nodes. This term is optimized jointly with the supervised and adaptation losses, encouraging the model to focus on a compact set of useful static dimensions rather than treating all feature dimensions equally. During target-year updating, the mask is learned only from the training graph constructed with the available labeled memory, so fixed target-test APKs do not influence feature-mask learning.

3.4.2. Attention-based GraphSAGE aggregation

After feature masking, AdaGuard employs an attention-based GraphSAGE encoder to learn node representations on the temporally isolated static semantic graph, combining neighborhood sampling and aggregation (Hamilton et al., 2017) with attention-weighted message passing (Veličković et al., 2018). We initialize the representation of node v as $h_v^{(0)} = \hat{x}_v$, where the superscript denotes the GraphSAGE layer index. For layer $\ell = 1, \ldots, L$, AdaGuard samples neighbors and aggregates their representations to update $h_v^{(\ell)}$.

For scalable graph learning, AdaGuard samples neighbors for each node at every GraphSAGE layer. The sampling probability is proportional to the edge weights defined in Section 3.3. Therefore, APK-to-cluster invocation weights and cluster-to-cluster semantic weights both affect which neighbors are more likely to participate in message passing.

Let $S(v)$ denote the sampled neighbor set of node v. At layer ℓ, AdaGuard uses an attention scoring function $q_\ell(\cdot)$ to measure the importance of each sampled neighbor. The normalized attention weight of neighbor u is computed as

$$a_{vu}^{(\ell)} = \frac{\exp(q_\ell(h_v^{(\ell-1)}, h_u^{(\ell-1)}))}{\sum_{r\in S(v)}\exp(q_\ell(h_v^{(\ell-1)}, h_r^{(\ell-1)}))}. \tag{16}$$

The neighbor representation of node v is obtained by attention-weighted aggregation:

$$n_v^{(\ell)} = \sum_{u\in S(v)} a_{vu}^{(\ell)} W_n^{(\ell)} h_u^{(\ell-1)}, \tag{17}$$

where $W_n^{(\ell)}$ is the neighbor transformation matrix at layer ℓ. The representation of node v is then updated by combining its self representation and the aggregated neighbor representation:

$$h_v^{(\ell)} = \phi(W_s^{(\ell)} h_v^{(\ell-1)} + n_v^{(\ell)}), \tag{18}$$

where $W_s^{(\ell)}$ is the self-node transformation matrix and $\phi(\cdot)$ denotes the nonlinear activation function. Dropout is applied during training for regularization. In our implementation, the static encoder contains two attention-based GraphSAGE layers with output dimensions of 256 and 2, respectively, and samples 5 and 10 neighbors at the two layers. The first layer maps the masked node features to a 256-dimensional representation with ReLU activation, whereas the second layer uses no activation and directly outputs the two logits for benign and malware classification. Dropout with a rate of 0.5 is applied during training. Thus, $L = 2$, $z_u^{\text{stat}} = h_u^{(2)}$, and the static prediction is computed as $p_u^{\text{stat}} = \text{softmax}(z_u^{\text{stat}})$. The static prediction p_u^{stat} is later used by the confidence-based late-fusion strategy described in Section 3.6.

3.5. Dynamic runtime-evidence encoder

After runtime tokens are extracted and mapped to token IDs, AdaGuard uses a gated runtime-evidence encoder to obtain the dynamic prediction. Inspired by permutation-invariant set encoding (Zaheer et al., 2017), this encoder treats runtime tokens as a set of behavioral evidence rather than a strictly ordered execution sequence, and aggregates valid token representations with a gated set-based structure. This design is suitable for dynamic reports whose behaviors may be organized by event fields rather than recorded as a precise execution trace. For APK u, let $e_{u,i}$ denote the embedding of the ith runtime token after token-ID lookup. AdaGuard uses two parallel transformations to compute the token feature and its gate:

$$f_{u,i} = \text{ReLU}(W_f e_{u,i} + b_f), \qquad g_{u,i} = \sigma(W_g e_{u,i} + b_g), \tag{19}$$

where $f_{u,i}$ is the transformed token feature, $g_{u,i}$ is the learned gate, W_f, W_g, b_f, and b_g are learnable parameters, and $\sigma(\cdot)$ denotes the sigmoid function. The gated token feature is then computed as

$$\bar{f}_{u,i} = f_{u,i} \odot g_{u,i}, \tag{20}$$

where $\odot$ denotes element-wise multiplication. The gate controls how much information each token dimension contributes to the dynamic representation. To aggregate token-level evidence, AdaGuard combines max pooling and sum pooling over valid runtime tokens. Max pooling captures the strongest runtime signal among tokens, while sum pooling preserves the overall strength of accumulated runtime behaviors. Padding positions are ignored during aggregation. The dynamic representation of APK u is computed as

$$z_u^{dyn} = \text{MaxPool}\left(\bar{f}_{u,1}, \ldots, \bar{f}_{u,M}\right) \oplus \text{SumPool}\left(\bar{f}_{u,1}, \ldots, \bar{f}_{u,M}\right), \tag{21}$$

where M is the maximum token length, $\oplus$ denotes vector concatenation, and z_u^{dyn} is the aggregated dynamic representation. In our implementation, $M = 150$. The dynamic representation is passed through batch normalization, dropout, and a linear classifier to produce the dynamic output vector. The dynamic prediction is then computed as $p_u^{dyn} = \text{softmax}(z_u^{dyn})$. This prediction provides complementary runtime evidence and is later used by the confidence-based late-fusion strategy in Section 3.6.

3.6. Confidence-based late fusion

The static graph channel and the dynamic runtime-evidence channel provide complementary information for Android malware detection. The static channel produces full-coverage predictions for all APKs and is relatively stable when runtime traces are noisy or unavailable, while the dynamic channel can provide additional runtime evidence for samples whose static predictions are uncertain. Therefore, AdaGuard adopts a confidence-based late-fusion strategy: confident static predictions are preserved, whereas dynamic evidence is incorporated only for uncertain static predictions. Let p_u^{stat} denote the probability vector produced by the static channel for APK u. The static confidence is computed as the maximum class probability in p_u^{stat}. If the static confidence is higher than a threshold τ_f, AdaGuard directly uses the static output. Otherwise, if dynamic evidence is available, the calibrated dynamic probability $\tilde{p}_u^{\text{dyn}}$ is fused with the static probability. Before fusion, AdaGuard applies probability clipping followed by temperature smoothing based on temperature scaling (Guo et al., 2017) to reduce numerical instability and over-confident dynamic outputs.

$$p_u^{final} = \omega p_u^{stat} + (1-\omega)\tilde{p}_u^{dyn}, \tag{22}$$

where ω is the static fusion weight and $\tilde{p}_u^{dyn}$ denotes the calibrated dynamic probability produced by the dynamic runtime-evidence channel. If dynamic evidence is unavailable, AdaGuard falls back to the static prediction. The confidence threshold τ_f, fusion weight ω, and calibration parameters are determined according to the validation protocol and are not tuned on fixed target-test samples. Algorithm 1 summarizes the complete inference procedure.

Algorithm 1 AdaGuard inference with confidence-based late fusion

Input: target APK u, static expert F_{stat}, dynamic expert F_{dyn}, threshold τ_f, static weight ω, temperature κ, clipping bound ϵ_{clip}
Output: predicted label $\hat{y}_u$
Definitions: p_u^{stat}, p_u^{dyn}, and p_u^{final} are the static, dynamic, and final probability vectors of APK u
Definitions: $\text{Clip}(p, \epsilon_{\text{clip}}, 1-\epsilon_{\text{clip}})$ clips each element of p into $[\epsilon_{\text{clip}}, 1-\epsilon_{\text{clip}}]$
Definitions: $\Gamma(\boldsymbol{p}; \kappa) = \text{softmax}(\log(\boldsymbol{p})/\kappa)$ denotes temperature smoothing
$p_u^{\text{stat}} \leftarrow F_{\text{stat}}(u)$
$C_u^{\text{stat}} \leftarrow \max_{i\in\{0,1\}} p_{u,i}^{\text{stat}}$
if $C_u^{\text{stat}} > \tau_f$ **then**
    $p_u^{\text{final}} \leftarrow p_u^{\text{stat}}$
else if dynamic evidence of u is unavailable **then**
    $p_u^{\text{final}} \leftarrow p_u^{\text{stat}}$
else
    $p_u^{\text{dyn}} \leftarrow F_{\text{dyn}}(u)$
    $\bar{p}_u^{\text{dyn}} \leftarrow \text{Clip}\left(p_u^{\text{dyn}}, \epsilon_{\text{clip}}, 1-\epsilon_{\text{clip}}\right)$
    $\tilde{p}_u^{\text{dyn}} \leftarrow \Gamma\left(\bar{p}_u^{\text{dyn}}; \kappa\right)$
    $p_u^{\text{final}} \leftarrow \omega p_u^{\text{stat}} + (1-\omega)\tilde{p}_u^{\text{dyn}}$
end if
$\hat{y}_u \leftarrow \arg\max_{i\in\{0,1\}} p_{u,i}^{\text{final}}$
return $\hat{y}_u$

Here, $U_u^{\text{stat}} = 1 - C_u^{\text{stat}}$ is used as an operational routing uncertainty score rather than as a formal estimate of epistemic uncertainty. Before fusion, the dynamic probability is clipped and temperature-smoothed to limit extreme confidence, while its contribution to the final prediction is further bounded by the fusion weight $1-\omega$. All fusion parameters are selected using validation data and remain fixed during testing. The reliability of the dynamic probabilities, the effectiveness of the confidence-based routing criterion, and the robustness of the selective-fusion mechanism under incomplete runtime evidence are further evaluated in Section 4.2.3.

3.7. Memory-anchored temporal adaptation

Under chronological concept drift, updating a detector with limited labeled target-year samples may lead to unstable adaptation. Although accumulated historical labeled memory provides useful past knowledge, the newly labeled target-year subset is usually small and may be distributionally biased. As a result, the representation space and decision boundary may become overly sensitive to limited target-year labels during year-by-year updating. To address this issue, AdaGuard performs memory-anchored temporal adaptation, where accumulated historical labeled memory and limited labeled target-year samples are jointly used to update the model in a controlled manner.

For target year t, AdaGuard incorporates the target-year training subset L_t^{tr} into the previous training memory to form the current labeled memory H_t, as defined in Section 3.1. Based on H_t, AdaGuard updates the static and dynamic experts separately. The static expert

is updated with supervised learning and memory-anchored regularization, including prototype alignment, knowledge distillation, and domain alignment. The dynamic expert is updated by expanding the runtime-token vocabulary using only available labeled training data and inheriting embeddings from the previous dynamic expert.

3.7.1. Static expert adaptation

Since AdaGuard updates the static expert using the full labeled memory H_t, catastrophic forgetting is less severe than in incremental learning settings where historical data are unavailable. However, the labeled target-year subset is still limited, and its feature distribution may differ from historical data. Therefore, the goal of static expert adaptation is not only to fit the available labeled samples, but also to stabilize the representation space and decision boundary during year-by-year updating.

To optimize the static expert at target year t, AdaGuard uses the phase-specific training graph induced by the labeled memory H_t and minimizes the following joint objective:

$$\mathcal{L}_s = \mathcal{L}_c + \lambda_m \mathcal{L}_m + \lambda_e \mathcal{L}_e + \lambda_k \mathcal{L}_k + \lambda_g \mathcal{L}_g, \tag{23}$$

where $\mathcal{L}_s$ is the overall static adaptation loss, $\mathcal{L}_c$ is the supervised cross-entropy loss (Goodfellow et al., 2016) on labeled samples in H_t, $\mathcal{L}_m$ is the feature-mask regularization term defined in Section 3.4, $\mathcal{L}_e$ is the elastic prototype alignment loss, $\mathcal{L}_k$ is the knowledge distillation loss from the previous static expert, and $\mathcal{L}_g$ is the gradient-reversal-based domain alignment loss. The coefficients λ_m, λ_e, λ_k, and λ_g control the contributions of the corresponding regularization terms.

Let z_u denote the static output representation of APK u. AdaGuard constructs a set of historical malicious prototypes $\mathcal{M} = \{\mu_1, \mu_2, \ldots, \mu_K\}$ from labeled malicious samples in the historical memory. Before updating target year t, up to 512 malicious samples are randomly selected from H_{t-1} and represented by the previously selected static expert. These representations are grouped using K-means, with $K = 5$ in our implementation, and the resulting cluster centers are used as the historical malicious prototypes. The prototypes remain fixed during the current-year update and are recomputed from the expanded historical memory before the next target-year update. During target-year updating, the elastic prototype alignment loss is applied only to malicious samples in the current mini-batch:

$$\mathcal{L}_e = \frac{1}{|B^+|} \sum_{u \in B^+} \min_{\mu_j \in \mathcal{M}} \left\| z_u - \mu_j \right\|_2, \tag{24}$$

where $B^+ = \{u \in B \mid y_u = 1\}$ denotes the malicious samples in mini-batch B. If $B^+ = \emptyset$, the prototype-alignment term is set to $\mathcal{L}_e = 0$ for that mini-batch and does not contribute to the overall objective. This loss softly pulls current malicious samples toward the closest historical malicious prototype, which helps stabilize the malware representation space under limited target-year labels.

The knowledge distillation term (Hinton et al., 2015) uses the previous static expert as a teacher. To preserve historical decision consistency without constraining adaptation to newly labeled target-year applications, the distillation loss is applied only to historical samples in the current mini-batch. Let $B_{\text{hist}} = B \cap H_{t-1}$ denote the historical-memory samples contained in mini-batch B. Let p_u^{old} and p_u^{new} denote the probability vectors produced by the previous and current static experts for APK $u \in B_{\text{hist}}$, respectively. The distillation loss is defined as

$$\mathcal{L}_k = \frac{1}{|B_{\text{hist}}|} \sum_{u \in B_{\text{hist}}} \text{KL}\left(p_u^{old} | p_u^{new}\right), \tag{25}$$

where $\text{KL}(\cdot|\cdot)$ denotes the Kullback–Leibler divergence. If $B_{\text{hist}} = \emptyset$, the distillation term is set to $\mathcal{L}_k = 0$ for that mini-batch and does not contribute to the overall objective. This term encourages the updated static expert to maintain prediction consistency with the previous expert on historical-memory samples. No distillation constraint is imposed on the newly labeled target-year samples in L_t^{tr}, because the previous expert may provide outdated predictions for applications affected by temporal concept drift. Therefore, the supervised classification loss drives adaptation to the current target year, while the distillation loss helps preserve historical decision consistency.

For domain alignment, AdaGuard uses a gradient reversal layer (Ganin et al., 2016) with a domain discriminator. Let $d_u \in \{0, 1\}$ denote the domain label of APK u, where $d_u = 0$ indicates historical memory samples and $d_u = 1$ indicates labeled target-year samples. Given the static representation z_u, the domain loss is

$$\mathcal{L}_g = -\frac{1}{|B|} \sum_{u \in B} \left[d_u \log D(z_u) + (1 - d_u) \log(1 - D(z_u)) \right], \tag{26}$$

where $D(\cdot)$ is the domain discriminator. The discriminator learns to distinguish historical and target-year samples, while the gradient reversal layer encourages the static encoder to learn representations that are less sensitive to year-specific distribution differences.

Together, $\mathcal{L}_e$, $\mathcal{L}_k$, and $\mathcal{L}_g$ regularize the static expert during temporal updating. They do not replace the supervised loss; instead, they help the static expert adapt to the current target year while keeping the representation space and decision boundary stable.

3.7.2. Dynamic expert adaptation

The dynamic expert is updated separately from the static expert. Its runtime-token vocabulary is initialized from the available source-year training samples and is expanded during temporal adaptation using only the available labeled training samples in the current phase. When new runtime tokens are added, AdaGuard enlarges the embedding layer and copies the embeddings of previously learned tokens from the previous dynamic expert. For each new runtime token, AdaGuard searches the previous vocabulary for the most similar historical token using the character-sequence similarity implemented by Python's `difflib.get_close_matches`. Specifically, let χ_{new} denote a newly introduced token and χ_{hist} denote a token in the previous vocabulary. The underlying `SequenceMatcher` ratio is computed as

$$\text{sim}_{\text{tok}}\left(\chi_{\text{new}}, \chi_{\text{hist}}\right) = \frac{2N_{\text{match}}\left(\chi_{\text{new}}, \chi_{\text{hist}}\right)}{|\chi_{\text{new}}| + |\chi_{\text{hist}}|}, \tag{27}$$

where $N_{\text{match}}(\chi_{\text{new}}, \chi_{\text{hist}})$ denotes the total number of characters contained in the matching blocks between the two token strings, and $|\cdot|$ denotes the string length. In our implementation, `get_close_matches` is configured with $n = 1$ and a cutoff of 0.65. Among the historical tokens satisfying this cutoff, only the one with the highest sequence-similarity score is returned for embedding initialization. If a match is found, its embedding is used to initialize the new token with a small Gaussian perturbation of standard deviation 0.01; otherwise, the default initialization of the expanded embedding layer is retained. Runtime tokens that are still outside the current vocabulary are mapped to the shared unknown token during inference. The dynamic expert is trained on applications with valid dynamic traces using the gated set-based encoder described in Section 3.5. To handle class imbalance, AdaGuard uses a class-weighted cross-entropy loss for dynamic expert training.

3.7.3. Overall temporal updating procedure

Algorithm 2 summarizes the overall temporal updating procedure of AdaGuard. For each target year, AdaGuard selects a limited labeled subset from the candidate pool under the annotation budget, splits the selected subset into training and validation parts, and updates the labeled memory. The static expert is updated on the phase-specific training graph with the memory-anchored objective, while the dynamic expert is updated through runtime-token vocabulary expansion and dynamic expert training. During each updating phase, the static and dynamic experts are evaluated on the accumulated validation memory V_t after each training epoch, and the checkpoint achieving the highest validation F1-score is retained. Early stopping is determined by the

number of consecutive epochs without improvement in validation F1-score. For the static expert, early stopping is enabled after a minimum training period of 100 epochs and uses a patience of 15 epochs. The dynamic expert uses a patience of 50 epochs. After the two experts are updated, the fixed target-year test set is evaluated only by inference using the confidence-based late-fusion strategy in Algorithm 1.

Algorithm 2 AdaGuard temporal adaptation under target-year label budgets

Input: source training set D_0^{tr}, source validation set D_0^{val}, label budget ρ, candidate pool $\{C_t\}_{t=1}^{T}$ and fixed test set $\{D_t^{\mathrm{test}}\}_{t=1}^{T}$
Output: updated AdaGuard states and predictions on fixed test sets
Train initial static expert F_{stat}^{0} using D_0^{tr}
Train initial dynamic expert F_{dyn}^{0} using dynamic traces from D_0^{tr}
Initialize labeled memory $H_0 \leftarrow D_0^{\mathrm{tr}}$
for each target year $t = 1, 2, \ldots, T$ **do**
    Select labeled subset $L_t^{\rho} \subset C_t$ under budget ρ
    Split L_t^{ρ} into training subset L_t^{tr} and validation subset L_t^{val}
    Update labeled memory $H_t \leftarrow H_{t-1} \cup L_t^{\mathrm{tr}}$
    Build the phase-specific static training graph using H_t
    Recompute historical malicious prototypes from H_{t-1} using F_{stat}^{t-1}
    Update F_{stat}^{t} from F_{stat}^{t-1} using Eq. (23)
    Expand the runtime-token vocabulary using dynamic traces from H_t
    Update F_{dyn}^{t} from F_{dyn}^{t-1} using valid dynamic traces from H_t
    Select the updated experts using $D_0^{\mathrm{val}} \cup L_1^{\mathrm{val}} \cup \cdots \cup L_t^{\mathrm{val}}$
    Predict samples in D_t^{test} using Algorithm 1
end for
return updated AdaGuard states and predictions on $\{D_t^{\mathrm{test}}\}_{t=1}^{T}$

4. Evaluation

4.1. Experimental design

In this section, we first overview the **Research Questions (RQs)**. Then, we present the details about the dataset and empirical setup. Last, we define the metrics we use for evaluation.

4.1.1. RQs

In this study, we aim to investigate the following RQs:

RQ1: To what extent do non-adaptive baseline detectors suffer from temporal model aging under chronological concept drift?

RQ2: Under the same target-year label budgets, can AdaGuard achieve stronger temporal adaptation than updated baselines?

RQ3: How reliable and effective is the confidence-based late-fusion strategy in identifying uncertain static predictions and incorporating available runtime evidence?

RQ4: How much does each key component contribute to the temporal robustness of AdaGuard?

RQ5: How compact and computationally scalable is the APK–API-cluster semantic graph during graph construction and temporal updating?

4.1.2. Data collection

We collect Android applications from the AndroZoo (Allix et al., 2016) repository, covering the period from 2013 to 2017. For malware labeling, we use the VirusTotal detection reports recorded in the AndroZoo metadata, together with the corresponding per-APK scan dates. APKs detected by no antivirus engine are labeled as benign, whereas APKs detected by at least two engines are labeled as malware; APKs with exactly one detection are excluded. These labels are based on the VirusTotal reports stored in the AndroZoo metadata used for dataset construction, rather than on historical verdicts aligned with each APK's release year or on newly queried VirusTotal reports. The VirusTotal information is used only to assign malware/benign labels, while the chronological order of APKs is determined independently from the APK-internal timestamp. The AndroZoo metadata snapshot used for dataset construction was downloaded on November 15, 2025. Specifically, the APK `dex_date` is used for chronological partitioning, whereas the recorded VirusTotal scan date is treated as a separate metadata field. Detailed per-year scan-date statistics for both the main and recent evaluation datasets are provided in Table 18.

Table 1
Overview of the temporal dataset (2013–2017).

Year	2013	2014	2015	2016	2017	Total
Malware	1542	1624	1609	1934	1918	8627
Benign	2810	2742	2891	2938	2780	14,161
Total	4352	4366	4500	4872	4698	22,788

To support chronological concept-drift evaluation, we arrange the applications chronologically according to the timestamp stored in each APK's DEX file, as provided by AndroZoo. To avoid direct duplication across temporal partitions, exact duplicate APKs are removed according to their SHA256 hashes before chronological splitting. Non-identical but highly similar APKs are intentionally retained, and no additional near-duplicate filtering based on an APK-similarity threshold is applied. Such APKs may correspond to different application versions or malware variants appearing across years and therefore represent part of the temporal evolution that this study aims to evaluate. Removing them solely because of high similarity could discard legitimate temporal variations and artificially alter the chronological data distribution. Samples with invalid or ambiguous timestamps are excluded. We then group the remaining applications by year, resulting in a five-year dataset from 2013 to 2017. For each application, AdaGuard extracts two types of evidence according to the feature construction procedure described in Section 3: static graph features and dynamic runtime-token traces. Table 1 summarizes the number of benign and malicious applications in each year.

4.1.3. Experimental settings and baseline methods

To evaluate the model's ability to resist temporal concept drift, we partition the dataset into five yearly time windows: 2013 serves as the source year, and 2014–2017 serve as four consecutive target years. The source year provides a fully labeled training set for initial model construction. For each target year, we first isolate 20% of the samples as a fixed test set, which remains completely unseen during any temporal adaptation process. The remaining 80% forms a candidate pool, from which only a limited proportion of samples can be annotated and used for model updating. The selected labeled subset is further split into 80% for training and the remaining 20% for validation. The training part is merged with historical training memory for expert updating, while the validation part is used for model selection and early stopping. After each target-year adaptation phase, the model is evaluated on the fixed test set from the same year, ensuring a fair and consistent assessment of detection performance on unseen target-year applications.

We compare AdaGuard against three representative baselines covering established machine learning approaches to Android malware detection, including traditional static-feature-based detection, abstract behavior modeling, and rejection based concept drift mitigation (Guerra-Manzanares, 2024). To ensure faithful and reproducible comparisons, all baselines are implemented based on the original open-source code released by their authors. We only adapt their input/output interfaces to our temporal dataset and evaluation protocol, while preserving their core feature extraction procedures and decision mechanisms. For fairness, the classifier-related hyperparameters of all baselines are tuned under the same validation protocol.

Drebin-SVM (Arp et al., 2014) is a binary linear SVM trained on Drebin-style static features and serves as the classical static-feature baseline. **MaMaDroid** (Onwuzurike et al., 2019) extracts API paths from the API call graph and abstracts these calls into higher-level representations, such as package, class, or family. It then models the abstracted API sequences as a Markov chain, computing the transition

Table 2
Summary of baselines and update protocols.

Method	Budget	Update protocol
Drebin-SVM	0%	Source-only static classifier
MaMaDroid-Fam	0%	Source-only family-level Markov model
TRANSCENDENT	0%	Source-only conformal rejection
Drebin-SVM-Upd	5/10/15%	SVM retraining
MaMaDroid-Fam-Upd	5/10/15%	Classifier retraining with family-level Markov features
TRANSCENDENT-Upd	5/10/15%	SVM retraining and rejection threshold recalibration
AdaGuard	5/10/15%	Memory-anchored adaptation with the same target-year label samples

Table 3
Actual numbers of target-year labeled samples under different label budgets. Each budget entry reports training/validation APKs.

Year	Candidate pool	5%	10%	15%
2014	3492	139/35	279/70	418/105
2015	3600	144/36	288/72	432/108
2016	3897	155/39	311/78	467/117
2017	3758	149/38	300/75	450/113
Total	14,747	587/148	1178/295	1767/443

Table 4
Evaluation metrics used in this study.

Metrics	Abbreviation	Meaning
True Positive	TP	Malware correctly identified as malware.
True Negative	TN	Benign correctly identified as benign.
False Positive	FP	Benign incorrectly identified as malware.
False Negative	FN	Malware incorrectly identified as benign.
Accepted F1	Accepted-F1	F1-score on accepted samples.
Rejection Rate	RR	rejected samples/total samples
Accuracy	Acc	(TP+TN)/(TP+TN+FP+FN)
Precision	P	TP/(TP+FP)
Recall	R	TP/(TP+FN)
F1-score	F1-score	2*P*R/(P+R)
Coverage-aware F1	Cov-F1	(1-RR)*Accepted-F1

probabilities between nodes to construct the static feature vector. In our evaluation, we realize MaMaDroid using its official source code and choose Random Forest (RF) as the detection model. To evaluate its structural robustness, we operate it in the family mode, resulting in a 122-dimensional feature vector in our implementation. Furthermore, the RF classifier is optimized on the source-domain training set to ensure a fair and competitive baseline. **TRANSCENDENT** (Barbero et al., 2022) is a rejection-based framework for mitigating concept drift in malware classification. Unlike ordinary binary classifiers, it does not necessarily assign a benign or malicious label to every test sample. Instead, it evaluates the reliability of each prediction made by an underlying classifier and rejects samples that are likely to be unreliable under distribution shift. In our evaluation, we realize TRANSCENDENT using its official source code and adopt the Drebin-feature binary SVM as its base classifier, following the Android malware setting in the original paper. The base SVM is optimized on the source-domain training set to ensure a fair and competitive baseline. For rejection thresholding, we follow the Q1 calibration rule used in TRANSCENDENT, where the first quartile of the p-values of correctly classified calibration samples is used as the rejection threshold. In the updated setting, denoted TRANSCENDENT-Upd, the underlying SVM is retrained using the historical labeled training samples and the same budgeted target-year training samples, while the Q1 rejection thresholds are recalibrated using the currently available validation data.

To fairly evaluate temporal adaptation, the non-adaptive baselines are first evaluated in a source-only setting, where each method is trained only on the 2013 source-domain data and directly tested on future years. This setting is used to examine the temporal aging behavior of existing detectors without target-domain updates. In the updated setting, denoted by the suffix "-Upd", we consider three target-year label budgets of 5%, 10%, and 15% of the target-year candidate pool. For each target year, the candidate pool is randomly permuted using a fixed selection seed, and the three budget-specific labeled subsets are obtained as progressively longer prefixes of the same randomized ordering. Therefore, the selected labeled subsets are nested, $L_t^{5\%} \subset L_t^{10\%} \subset L_t^{15\%}$, rather than independently sampled. For each budget, the same selected labeled samples are shared by all updated methods. Specifically, Drebin-SVM-Upd retrains the linear SVM, MaMaDroid-Fam-Upd retrains the backend classifier using family-level Markov features, and TRANSCENDENT-Upd retrains the underlying SVM and recalibrates the Q1 rejection thresholds. AdaGuard is updated using the same randomly selected labeled samples as the updated baselines, and performs memory-anchored temporal adaptation. This protocol controls for method-specific differences in sample selection and enables a fair comparison under the same target-year label budget. Table 2 summarizes the target-label usage and update mechanism of all baseline methods and AdaGuard evaluated in this study. Table 3 reports the exact numbers of labeled APKs used for target-year training and validation. The label-budget percentages are defined with respect to the candidate pool rather than the complete target-year dataset.

To evaluate the robustness of AdaGuard against randomness introduced by data sampling and model training, we perform repeated experiments with independent random seeds. For each repeated run, all compared methods share the same random configuration to ensure a fair comparison. When random labeled samples are involved, the target-year labeled subsets and their corresponding training/validation partitions are regenerated, while the fixed target-year test sets remain unchanged. The corresponding results are reported as mean ± standard deviation in the following sections.

4.1.4. Metrics

To comprehensively assess the performance of our proposed Android malware detection model, we adopt several widely used classification metrics, which are summarized in Table 4. Accuracy measures the overall correctness of the classifier, the ratio of correctly identified samples to the total number of samples. Precision indicates the proportion of true malware among all applications that are predicted as malicious, reflecting the reliability of positive predictions. Recall quantifies the model's ability to correctly detect actual malware instances, namely the fraction of malware samples that are successfully identified. Finally, the F1-score is the harmonic mean of Precision and Recall, providing a balanced measure that accounts for both false positives and false negatives. A higher F1-score implies better overall detection capability, especially when the dataset is imbalanced.

Since TRANSCENDENT and TRANSCENDENT-Upd are classifiers with a rejection option, they may reject uncertain samples rather than assign a benign or malicious label to every test application. Therefore, standard full-test-set metrics alone cannot fully characterize their operational behavior. For these rejection-based baselines, we additionally report Accepted F1, Rejection Rate, and Coverage-aware F1. Accepted F1 evaluates the detection performance only on the samples accepted by the rejection mechanism. Rejection Rate measures the proportion of test samples that are rejected or quarantined. Coverage-aware F1 further reflects the trade-off between accepted-sample reliability and decision coverage. For ordinary classifiers without rejection, Coverage-aware F1 is equivalent to the standard F1-score.

Table 5
Source-only F1-scores of full-coverage baselines on fixed yearly test sets.

Method	2013	2014	2015	2016	2017	Avg.$_{tgt}$
Drebin-SVM	0.7756	0.6464	0.6521	0.5174	0.4348	0.5627
MaMaDroid-Fam	0.6795	0.5629	0.4304	0.4229	0.1878	0.4010

Table 6
Source-only rejection-aware performance of TRANSCENDENT on fixed yearly test sets.

Metrics	2013	2014	2015	2016	2017	Avg.$_{tgt}$
Accepted-F1	0.8728	0.7625	0.7417	0.6493	0.3943	0.6370
RR	31.8%	38.7%	53.8%	62.3%	60.3%	53.8%
Cov-F1	0.5953	0.4674	0.3427	0.2448	0.1566	0.3029

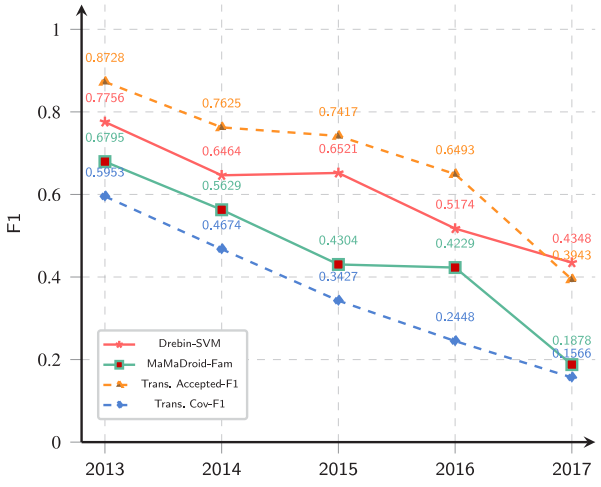


Fig. 3. Source-only temporal aging trends of non-adaptive baselines. Drebin-SVM and MaMaDroid-Fam are reported by full-test-set F1, whereas TRANSCENDENT is reported by Accepted-F1 and Coverage-aware F1 due to its rejection option.

4.2. Experimental results and analysis

4.2.1. RQ1: Temporal aging of non-adaptive baselines

To answer RQ1, we first evaluate whether non-adaptive detectors suffer from temporal model aging under chronological deployment. In this setting, each baseline is trained only on the 2013 source-year data and is then directly tested on the fixed test sets from 2014 to 2017 without using any target-year labels for updating. Table 5 reports the full-test-set F1-scores of Drebin-SVM and MaMaDroid-Fam, while Table 6 reports the Accepted F1, Rejection Rate, and Coverage-aware F1 of TRANSCENDENT. Fig. 3 further visualizes the source-only aging trends across years. Hereafter, Avg.$_{tgt}$ denotes the arithmetic mean of a given metric over the four target-year test sets from 2014 to 2017.

As shown in Table 5 and Fig. 3, both full-coverage baselines exhibit clear temporal aging when deployed without target-year adaptation. Drebin-SVM decreases from an F1-score of 0.7756 in the source year to 0.4348 in 2017, while MaMaDroid-Fam drops from 0.6795 to 0.1878 over the same period. Although the degradation is not strictly monotonic in every adjacent year, the overall downward trend is evident as the temporal gap from the source year increases. This result indicates that feature patterns learned from the 2013 source year become progressively less reliable for future Android applications. Compared with Drebin-SVM, MaMaDroid-Fam shows a more severe long-term decline, suggesting that coarse-grained API-family transition features alone are insufficient to preserve detection robustness under multi-year behavioral and API evolution.

Table 6 further shows that TRANSCENDENT exhibits a different aging behavior due to its rejection mechanism. Its Accepted F1 remains relatively high in some target years, reaching 0.7625 in 2014, 0.7417 in 2015, and 0.6493 in 2016. However, these values are computed only on the accepted samples and therefore do not reflect the full-test-set detection utility. As the temporal gap increases, TRANSCENDENT rejects a large proportion of test applications, with the Rejection Rate increasing from 38.7% in 2014 to 53.8% in 2015 and 62.3% in 2016. Consequently, its Coverage-aware F1 drops from 0.5953 in the source year to 0.1566 in 2017. This result indicates that rejection-based drift mitigation can preserve the reliability of accepted predictions, but it does so by sacrificing decision coverage. Therefore, Accepted F1 alone may overestimate the practical detection capability of rejection-based methods under long-term temporal drift.

These observations answer RQ1 by showing that non-adaptive detectors suffer substantial temporal model aging under chronological deployment. Full-coverage baselines gradually lose detection effectiveness as the target year moves farther away from the source year, while the rejection-based baseline maintains reliability on part of the accepted samples at the cost of reduced decision coverage. Therefore, relying only on source-year training is insufficient for long-term Android malware detection. This motivates the next experiment, where we evaluate whether different detectors can be effectively updated under the same limited target-year label budgets.

4.2.2. RQ2: Budget-matched temporal adaptation

To answer RQ2, we evaluate whether AdaGuard can achieve stronger temporal adaptation than updated baselines under the same target-year label budgets. For each target year, all updated methods use the same selected labeled subset from the candidate pool, while the fixed target-year test set is used only for final evaluation. We consider three label budgets 5%, 10%, and 15% of the target-year candidate pool. Table 7 reports the complete budget-matched adaptation results, Fig. 4 shows the year-wise Cov-F1 trends under each budget, and Fig. 5 summarizes the average target-year label efficiency across budgets. Table 8 further evaluates whether the observed performance advantage is robust to different random selections of labeled target-year samples.

Table 7 shows that AdaGuard consistently achieves the best average target-year performance under all label budgets. With only a 5% target-year label budget, AdaGuard obtains an Avg.$_{tgt}$ Cov-F1 of 0.7325, outperforming the best updated baseline, Drebin-SVM-Upd, by 0.06. When the budget increases to 10% and 15%, AdaGuard further improves the Avg.$_{tgt}$ Cov-F1 to 0.7658 and 0.7682, respectively, and still maintains clear advantages of 0.0467 and 0.0449 over the strongest baseline in each setting. These results indicate that memory-anchored temporal adaptation enables AdaGuard to make more effective use of limited target-year labels than conventional retraining-based baselines. In contrast, although TRANSCENDENT-Upd recalibrates its rejection thresholds with the available labeled data, its Cov-F1 remains lower because a considerable portion of target-year samples is rejected rather than automatically classified.

Fig. 4 provides a year-wise view of the adaptation results. Across all three label budgets, AdaGuard maintains a Cov-F1 above 0.7000 on every target year, indicating that its strong performance is not confined to a single year or budget setting. Under the 5% budget, AdaGuard achieves Cov-F1 values of 0.7138, 0.7129, 0.7005, and 0.8026 from 2014 to 2017, respectively. Its average target-year Cov-F1 further increases from 0.7325 under the 5% budget to 0.7658 and 0.7682 under the 10% and 15% budgets. The gain is particularly clear from 5% to 10%, whereas the improvement from 10% to 15% becomes marginal, suggesting diminishing returns from additional target-year labels. The improvement is not strictly monotonic for every individual year. For example, the 2014 Cov-F1 increases from 0.7138 to 0.7608 and then decreases slightly to 0.7410. Such local fluctuations are expected under finite-budget sampling because increasing the budget changes the composition of the labeled training and validation subsets, as well as the checkpoint selected through validation-based early stopping. Additional labeled samples therefore improve the overall trend but do not

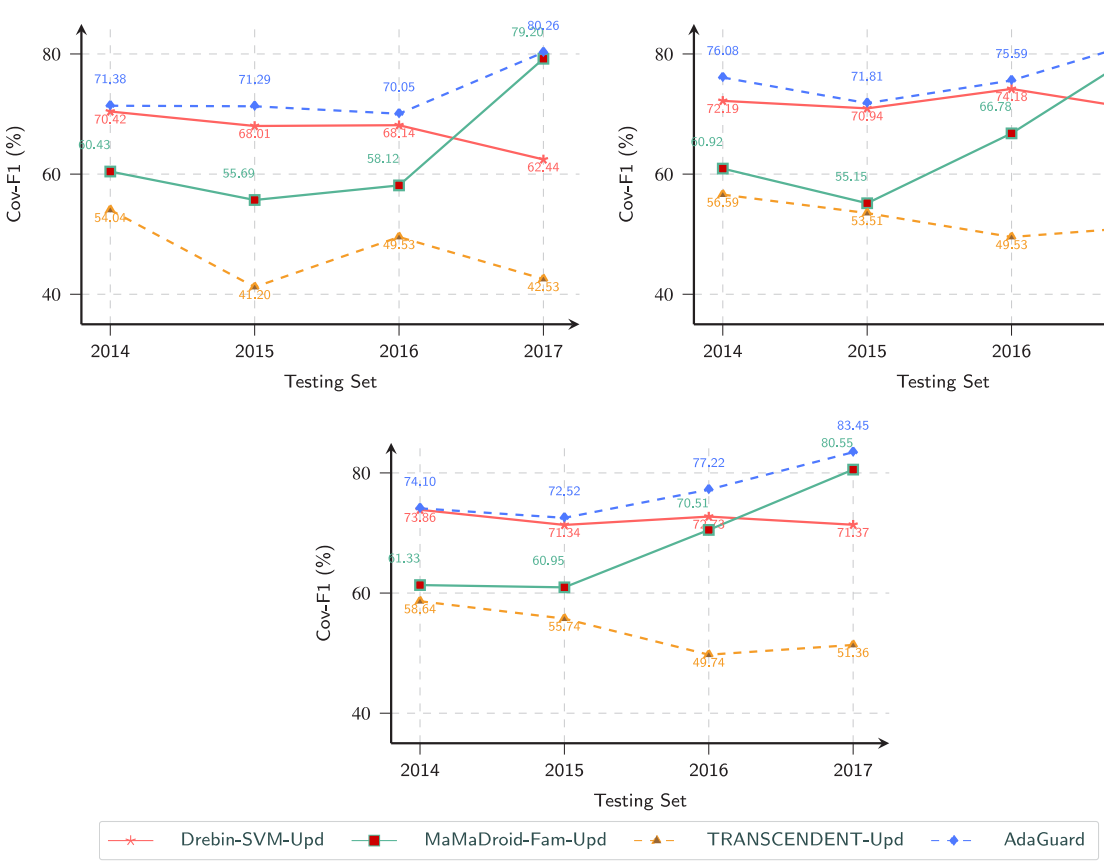


Fig. 4. Year-wise Coverage-aware F1 (Cov-F1) trends of updated methods under different target-year label budgets.

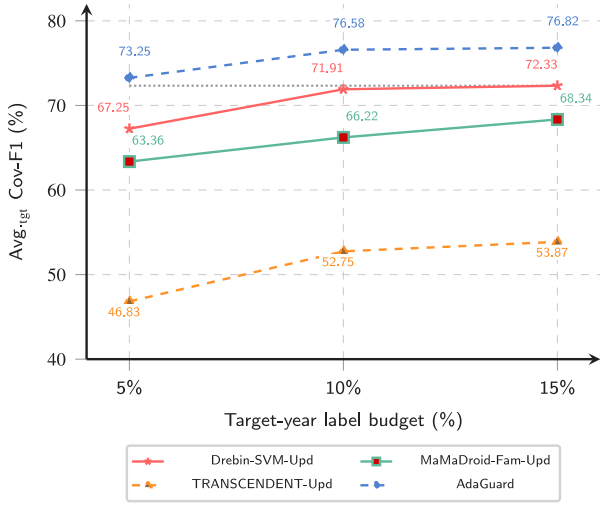


Fig. 5. Label efficiency of updated methods under different target-year label budgets.

guarantee a monotonic gain on every individual test year. The results also reveal markedly different method-specific responses to the 2017 test distribution. AdaGuard and MaMaDroid-Fam-Upd show substantial improvements from 2016 to 2017, whereas Drebin-SVM-Upd decreases across all three budgets and TRANSCENDENT-Upd remains comparatively limited. These opposite trends may reflect differences in how the underlying feature representations respond to year-specific concept drift. MaMaDroid-Fam-Upd models abstract API-level behaviors and may therefore align better with the behavioral patterns present in the 2017 applications, while Drebin-SVM-Upd relies on sparse static features whose vocabulary and prevalence can change considerably over time. For TRANSCENDENT-Upd, Cov-F1 is additionally affected by prediction coverage, so changes in the rejection rate may offset improvements on the accepted samples. AdaGuard's consistently strong 2017 results are instead consistent with its cumulative chronological adaptation and the complementary use of static and runtime evidence. Overall, although individual methods exhibit year-specific fluctuations, AdaGuard provides the strongest overall performance across budgets and target years under chronological concept drift.

Fig. 5 further compares the average target-year Cov-F1 under different label budgets. AdaGuard shows stronger label efficiency than all updated baselines. In particular, in the fixed-selection main comparison of Table 7, AdaGuard under the 5% label budget achieves an Avg._{tgt} Cov-F1 of 0.7325, slightly exceeding the strongest updated baseline under the 15% budget. However, the multi-seed analysis in Table 8 provides a more conservative assessment of this cross-budget comparison: AdaGuard with 5% labels obtains an average target-year F1-score of 0.7218, compared with 0.7224 for Drebin-SVM-Upd with 15% labels. Therefore, the multi-seed results do not support a strict cross-budget superiority claim. Instead, they show that AdaGuard can achieve nearly the same average performance while using only one-third of the target-year labeling budget, demonstrating strong label efficiency and competitive temporal adaptation performance. Moreover, AdaGuard improves from 0.7325 to 0.7658 when the budget increases from 5% to 10%, while the gain from 10% to 15% becomes marginal. This suggests that the proposed memory-anchored adaptation can effectively exploit limited target-year labels, and that most of the adaptation benefit is already captured under a small label budget.

Table 7
Temporal adaptation results of AdaGuard and updated baselines under the same target-year label budgets.

Budget (%)	Method	Year	Performance (%)					
			Acc	P	R	F1-score	RR	Cov-F1
5	Drebin-SVM-Upd	2014	80.78	78.74	63.69	70.42	0.0	70.42
		2015	78.89	73.72	63.12	68.01	0.0	68.01
		2016	77.74	71.83	64.80	68.14	0.0	68.14
		2017	73.51	71.88	55.20	62.44	0.0	62.44
		$Avg._{tgt}$	**77.73**	**74.04**	**61.70**	**67.25**	**0.0**	**67.25**
	MaMaDroid-Fam-Upd	2014	76.77	77.89	49.36	60.43	0.0	60.43
		2015	74.89	74.74	44.37	55.69	0.0	55.69
		2016	76.21	82.14	44.97	58.12	0.0	58.12
		2017	84.57	85.71	73.60	79.20	0.0	79.20
		$Avg._{tgt}$	78.11	80.12	53.07	63.36	0.0	63.36
	TRANSCENDENT-Upd	2014	89.55	91.45	74.73	82.25	34.3	54.04
		2015	89.42	83.90	76.74	80.16	48.6	41.20
		2016	84.76	80.63	64.18	71.47	30.7	49.53
		2017	89.35	90.78	77.11	83.39	49.0	42.53
		$Avg._{tgt}$	88.27	86.69	73.19	79.32	40.65	46.83
	AdaGuard (ours)	2014	81.46	80.16	64.33	71.38	0.0	71.38
		2015	80.67	75.52	67.50	71.29	0.0	71.29
		2016	80.00	77.82	63.69	70.05	0.0	70.05
		2017	83.94	78.72	81.87	80.26	0.0	80.26
		$Avg._{tgt}$	**81.52**	**78.06**	**69.35**	**73.25**	**0.0**	**73.25**
	$Avg._{tgt}$ gain over best baseline		(↑3.79)	(↑4.02)	(↑7.65)	(↑6.00)	–	(↑6.00)
10	Drebin-SVM-Upd	2014	81.58	78.87	66.56	72.19	0.0	72.19
		2015	80.33	74.74	67.50	70.94	0.0	70.94
		2016	82.15	79.11	69.83	74.18	0.0	74.18
		2017	77.66	74.77	66.40	70.34	0.0	70.34
		$Avg._{tgt}$	**80.43**	**76.87**	**67.57**	**71.91**	**0.0**	**71.91**
	MaMaDroid-Fam-Upd	2014	76.66	76.44	50.64	60.92	0.0	60.92
		2015	74.33	72.82	44.37	55.15	0.0	55.15
		2016	79.49	82.38	56.15	66.78	0.0	66.78
		2017	86.28	85.96	78.40	82.01	0.0	82.01
		$Avg._{tgt}$	79.19	79.40	57.39	66.22	0.0	66.22
	TRANSCENDENT-Upd	2014	88.84	92.25	68.95	78.92	28.3	56.59
		2015	88.65	86.75	72.78	79.15	32.4	53.51
		2016	87.73	88.68	71.21	78.99	37.3	49.53
		2017	88.53	89.39	76.92	82.69	37.9	51.35
		$Avg._{tgt}$	88.44	89.27	72.47	79.94	33.98	52.75
	AdaGuard (ours)	2014	82.95	76.70	75.48	76.08	0.0	76.08
		2015	80.11	72.38	71.25	71.81	0.0	71.81
		2016	84.10	86.64	67.04	75.59	0.0	75.59
		2017	86.60	84.68	81.07	82.83	0.0	82.83
		$Avg._{tgt}$	**83.44**	**80.10**	**73.71**	**76.58**	**0.0**	**76.58**
	$Avg._{tgt}$ gain over best baseline		(↑3.01)	(↑3.23)	(↑6.14)	(↑4.67)	–	(↑4.67)
15	Drebin-SVM-Upd	2014	82.27	78.49	69.75	73.86	0.0	73.86
		2015	80.44	74.49	68.44	71.34	0.0	71.34
		2016	81.54	79.47	67.04	72.73	0.0	72.73
		2017	78.83	77.50	66.13	71.37	0.0	71.37
		$Avg._{tgt}$	**80.77**	**77.49**	**67.84**	**72.33**	**0.0**	**72.33**
	MaMaDroid-Fam-Upd	2014	77.35	79.29	50.00	61.33	0.0	61.33
		2015	76.22	73.25	52.19	60.95	0.0	60.95
		2016	81.13	82.71	61.45	70.51	0.0	70.51
		2017	85.00	83.43	77.87	80.55	0.0	80.55
		$Avg._{tgt}$	79.93	79.67	60.38	68.34	0.0	68.34
	TRANSCENDENT-Upd	2014	89.08	90.80	73.27	81.10	27.7	58.64
		2015	88.94	87.01	72.83	79.29	29.7	55.74
		2016	87.32	88.67	67.86	76.88	35.3	49.74
		2017	88.46	86.52	77.78	81.91	37.3	51.36
		$Avg._{tgt}$	88.45	88.25	72.94	79.80	32.5	53.87
	AdaGuard (ours)	2014	82.72	80.30	68.79	74.10	0.0	74.10
		2015	81.22	75.59	69.69	72.52	0.0	72.52
		2016	84.21	82.08	72.91	77.22	0.0	77.22
		2017	87.13	85.67	81.33	83.45	0.0	83.45
		$Avg._{tgt}$	**83.82**	**80.91**	**73.18**	**76.82**	**0.0**	**76.82**
	$Avg._{tgt}$ gain over best baseline		(↑3.05)	(↑3.42)	(↑5.34)	(↑4.49)	–	(↑4.49)

Table 8
Target-year F1 stability of AdaGuard and Drebin-SVM-Upd under different random selections of labeled samples. Results are reported as mean ± standard deviation over four random seeds.

Label budget	2014		2015		2016		2017		Average	
	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd
5%	0.6921 ± 0.0305	**0.7119 ± 0.0071**	**0.6880 ± 0.0271**	0.6757 ± 0.0208	**0.7228 ± 0.0155**	0.6729 ± 0.0141	**0.7844 ± 0.0162**	0.6354 ± 0.0370	**0.7218 ± 0.0124**	0.6740 ± 0.0057
10%	**0.7264 ± 0.0306**	0.7221 ± 0.0168	**0.7148 ± 0.0308**	0.6976 ± 0.0430	**0.7611 ± 0.0110**	0.7257 ± 0.0202	**0.8192 ± 0.0080**	0.7106 ± 0.0176	**0.7553 ± 0.0117**	0.7140 ± 0.0074
15%	**0.7447 ± 0.0062**	0.7272 ± 0.0127	**0.7176 ± 0.0165**	0.7114 ± 0.0244	**0.7683 ± 0.0032**	0.7324 ± 0.0090	**0.8269 ± 0.0074**	0.7183 ± 0.0119	**0.7644 ± 0.0048**	0.7224 ± 0.0052

Since the composition and class distribution of randomly selected labeled samples may affect adaptation performance, particularly under small label budgets, we further examine whether the observed advantage is robust to different labeled-data selections. Following the multi-seed protocol described in Section 4.1.3, we compare AdaGuard with Drebin-SVM-Upd, the strongest updated baseline in the main budget-matched comparison. Table 8 reports the resulting F1-scores as mean ± standard deviation over four independent runs. Since both methods provide full prediction coverage, their F1-score is equivalent to Cov-F1.

As shown in Table 8, the multi-seed results preserve the overall performance advantage observed in the main budget-matched comparison. Under the particularly challenging 5% label budget, AdaGuard achieves an average target-year F1-score of 0.7218 ± 0.0124, compared with 0.6740 ± 0.0057 for Drebin-SVM-Upd, corresponding to an average advantage of 4.78 percentage points. Under the 10% and 15% label budgets, AdaGuard achieves 0.7553 ± 0.0117 and 0.7644 ± 0.0048, respectively, whereas Drebin-SVM-Upd obtains 0.7140 ± 0.0074 and 0.7224 ± 0.0052. The corresponding average advantages remain 4.13 and 4.20 percentage points, respectively.

The year-wise results also confirm that random labeled-sample composition can affect individual outcomes, especially when the label budget is small. For example, Drebin-SVM-Upd obtains a higher mean F1-score than AdaGuard on the 2014 test set under the 5% budget. Nevertheless, AdaGuard achieves the higher mean F1-score in 11 of the 12 year-budget combinations and retains a clear average target-year advantage under all three label budgets. Moreover, the standard deviation of AdaGuard's average target-year F1-score decreases from 0.0124 under the 5% budget to 0.0117 and 0.0048 under the 10% and 15% budgets, respectively. These results show that although limited labeled-data sampling introduces measurable variability, the observed temporal-adaptation advantage of AdaGuard is not specific to a single favorable random selection of labeled samples.

Overall, the budget-matched comparison and the additional multi-seed analysis jointly answer RQ2 positively. Under the same target-year label budgets and fixed-test evaluation protocol, AdaGuard achieves the strongest average target-year performance in the main comparison and retains a clear average advantage over the strongest updated baseline across different random labeled-data selections. Although individual year-wise outcomes may vary under limited-data sampling, the multi-seed evaluation confirms that the overall advantage is not specific to a single favorable labeled-data partition. These results suggest that simply retraining conventional detectors or recalibrating rejection thresholds is insufficient to fully address chronological concept drift under limited annotations. In contrast, AdaGuard benefits from memory-anchored temporal adaptation and confidence-based late fusion, enabling it to exploit limited target-year labels more effectively while maintaining full-coverage prediction.

4.2.3. RQ3: Reliability and robustness of selective fusion

Runtime evidence is not always available in practical Android malware analysis. Some APKs may fail to install, launch, or exhibit sufficient behavior in a sandbox, preventing the collection of usable runtime logs. Methods that require complete views may therefore exclude applications for which the required dynamic features cannot be extracted. For example, EvolveDroid removes applications that cannot be processed by its required static or dynamic analysis tools during dataset construction (Huang et al., 2024). In contrast, AdaGuard retains such APKs and directly uses the static prediction when no usable runtime log is available.

To evaluate the proposed invocation strategy, we compare two late-fusion policies on the same fixed target-year test sets. Always-Fusion invokes the dynamic expert whenever a usable runtime log is available and combines the static and dynamic predictions through late fusion. Selective-Fusion invokes the dynamic expert only when the static confidence does not exceed the threshold τ_f; otherwise, the static prediction is retained. For APKs without usable runtime logs, both policies fall back to the static expert. Thus, the two policies differ only in the dynamic-expert invocation rule. The fusion threshold, static weight, and temperature were optimized on the validation set and fixed at $\tau_f = 0.944107$, $\omega = 0.761014$, and $\kappa = 2.0$, respectively.

Table 9 reports runtime-log availability and dynamic-expert invocation under the 5%, 10%, and 15% target-year label budgets. Among the 3689 APKs in the fixed test sets, 2421 have usable runtime logs, corresponding to an availability rate of 65.63%. The remaining 1268 APKs, accounting for 34.37% of the test set, are retained and classified through static fallback rather than being excluded from evaluation. AdaGuard therefore does not require usable runtime evidence for every APK. For APKs with usable runtime logs, Always-Fusion invokes the dynamic expert for all 2421 samples. Selective-Fusion performs 2157, 2296, and 2349 invocations under the 5%, 10%, and 15% label budgets, respectively. It therefore avoids 264, 125, and 72 invocations, corresponding to reductions of 10.90%, 5.16%, and 2.97% relative to Always-Fusion.

The reduction becomes smaller at higher label budgets because the same threshold is applied to static experts with different confidence distributions; a higher F1-score does not necessarily mean that more predictions exceed τ_f. The gate is intentionally conservative and bypasses the dynamic expert only for highly confident static predictions. Its objective is therefore not to maximize call reduction, but to avoid likely redundant invocations while limiting the risk of suppressing useful runtime evidence.

Notably, Selective-Fusion and Always-Fusion achieve identical average F1-scores under all three label budgets. This result follows from the decision-preserving property of the adopted confidence gate. Let $\hat{y}_u^{\mathrm{stat}} = \arg\max_{i\in\{0,1\}} p_{u,i}^{\mathrm{stat}}$ denote the class predicted by the static expert. According to Algorithm 1, the corresponding static confidence is $C_u^{\mathrm{stat}} = p_{u,\hat{y}_u^{\mathrm{stat}}}^{\mathrm{stat}}$. For an APK whose dynamic-expert invocation is skipped by Selective-Fusion, $C_u^{\mathrm{stat}} > \tau_f$. Let $\boldsymbol{p}_u^{\mathrm{AF}}$ denote the probability vector that would be produced by Always-Fusion for the same APK. Since the calibrated dynamic probabilities are nonnegative, the probability assigned to the static predicted class satisfies

$$\begin{aligned} p_{u,\hat{y}_u^{\mathrm{stat}}}^{\mathrm{AF}} &= \omega p_{u,\hat{y}_u^{\mathrm{stat}}}^{\mathrm{stat}} + (1-\omega)\tilde{p}_{u,\hat{y}_u^{\mathrm{stat}}}^{\mathrm{dyn}} \\ &\geq \omega p_{u,\hat{y}_u^{\mathrm{stat}}}^{\mathrm{stat}} = \omega C_u^{\mathrm{stat}} > \omega\tau_f \\ &= 0.761014 \times 0.944107 \approx 0.7185 > 0.5. \end{aligned} \tag{28}$$

Because $\boldsymbol{p}_u^{\mathrm{AF}}$ is a binary probability vector, assigning a probability greater than 0.5 to $\hat{y}_u^{\mathrm{stat}}$ guarantees that this class remains the final prediction under Always-Fusion. Therefore, for high-confidence APKs skipped by Selective-Fusion, invoking the dynamic expert under Always-Fusion cannot overturn the static prediction. For low-confidence APKs with usable runtime evidence, both policies apply

Table 9
Runtime-evidence availability and dynamic-expert invocation under different target-year label budgets.

Label budget	Test APKs	Runtime available	Static fallback	Always calls	Selective calls	Skipped calls	Reduction (%)	F1-score (%) Always/Selective
5%	3689	2421	1268	2421	2157	264	10.90	73.25/73.25
10%	3689	2421	1268	2421	2296	125	5.16	76.58/76.58
15%	3689	2421	1268	2421	2349	72	2.97	76.82/76.82

Note: Runtime available denotes APKs with usable runtime logs. Static fallback denotes APKs without usable runtime logs that are classified by the static expert. Reduction is calculated relative to Always-Fusion over APKs with usable runtime logs. The F1-score is averaged over the four fixed target-year test sets.

Table 10
Dynamic-probability reliability and static-confidence error discrimination under different target-year label budgets. The reported values are pooled over the 2014–2017 target-test sets and presented as mean ± standard deviation over five random seeds.

Label budget	Raw ECE↓	Smoothed ECE↓	Raw Brier↓	Smoothed Brier↓	Error AUROC↑	High-conf. error (%)	Uncertain error (%)	Error capture (%)
5%	0.0997 ± 0.0124	**0.0358 ± 0.0106**	0.2011 ± 0.0061	**0.1905 ± 0.0038**	0.719 ± 0.010	5.69 ± 1.46	21.89 ± 0.48	96.13 ± 1.97
10%	0.0651 ± 0.0233	**0.0491 ± 0.0198**	0.1851 ± 0.0071	**0.1823 ± 0.0046**	0.724 ± 0.009	3.99 ± 1.32	19.00 ± 0.83	98.02 ± 1.54
15%	0.0594 ± 0.0117	**0.0521 ± 0.0080**	0.1789 ± 0.0039	**0.1779 ± 0.0030**	0.729 ± 0.011	2.57 ± 0.98	18.05 ± 0.88	99.18 ± 0.88

the same late-fusion rule and therefore produce the same prediction. For APKs without usable runtime evidence, both policies fall back to the static prediction. Consequently, Selective-Fusion and Always-Fusion produce identical hard-label predictions for all test APKs under the adopted fusion parameters, which explains their identical F1-scores.

To further assess the reliability of the confidence-based late-fusion mechanism, we evaluate both the probabilistic reliability of the dynamic expert and the effectiveness of the static-confidence routing criterion. Expected Calibration Error (ECE) measures the discrepancy between predictive confidence and empirical accuracy, while the Brier score measures the squared error of probabilistic predictions; lower values indicate better probabilistic reliability. We further use error-detection AUROC to evaluate how effectively the routing uncertainty $U_u^{\mathrm{stat}} = 1 - C_u^{\mathrm{stat}}$ ranks erroneous static predictions above correct ones, where a higher value indicates better error discrimination. Error capture denotes the percentage of all static prediction errors that fall into the uncertain subset.

For each label budget, the reported metrics are first computed by pooling the corresponding target-test samples from 2014–2017 rather than by simply averaging the four year-wise metric values. The final results are reported as mean ± standard deviation over five independent random seeds. ECE and Brier score are evaluated on the 2421 APKs with usable runtime evidence, whereas the static-confidence diagnostics are evaluated on all 3689 target-test APKs.

As shown in Table 10, temperature smoothing consistently improves the probabilistic reliability of the dynamic expert across different label budgets. Under the 5%, 10%, and 15% label budgets, the average ECE decreases from 0.0997 ± 0.0124 to 0.0358 ± 0.0106, from 0.0651 ± 0.0233 to 0.0491 ± 0.0198, and from 0.0594 ± 0.0117 to 0.0521 ± 0.0080, respectively. The corresponding Brier scores also decrease under all three budgets, indicating that temperature smoothing provides more reliable probability estimates for the dynamic expert across different labeled-data availability settings.

The static-confidence routing criterion nevertheless shows consistent effectiveness in identifying higher-risk predictions. The error-detection AUROC values are 0.719 ± 0.010, 0.724 ± 0.009, and 0.729 ± 0.011 under the 5%, 10%, and 15% label budgets, respectively. Meanwhile, the uncertain subsets exhibit error rates of 21.89 ± 0.48%, 19.00 ± 0.83%, and 18.05 ± 0.88%, compared with only 5.69 ± 1.46%, 3.99 ± 1.32%, and 2.57 ± 0.98% for the corresponding high-confidence subsets. Therefore, uncertain samples remain substantially more error-prone than high-confidence samples across different label budgets. In addition, the conservative routing threshold captures more than 96% of static prediction errors under all settings, further supporting the use of static confidence as an operational criterion for selective runtime-evidence invocation.

To further examine the sensitivity of AdaGuard to incomplete runtime evidence, we conduct a controlled runtime-evidence degradation test under the intermediate 10% label-budget setting. The analysis is performed on the same 2421 test APKs for which usable runtime evidence is originally available. We randomly suppress 25%, 50%, and 75% of the runtime tokens after unigram recovery using three independent perturbation seeds, while keeping the trained models, static routing decisions, and all fusion parameters fixed. The clean condition with 0% suppression is used as the reference for measuring performance degradation. For each nonzero suppression level, the F1-score is first macro-averaged across the four target years for each perturbation seed, and the reported value is the mean ± standard deviation over the three seeds, as summarized in Table 11. Because the stress test is evaluated only on the runtime-available subset, its clean-condition F1-score differs slightly from the full-test-set result reported earlier, where all 3689 APKs are retained through static fallback.

As runtime evidence becomes increasingly incomplete, the dynamic expert experiences a clear degradation because its prediction directly relies on the available runtime tokens. Its macro-averaged F1-score decreases from 0.7069 under the clean condition to 0.6897 ± 0.0064, 0.6413 ± 0.0127, and 0.5669 ± 0.0053 when 25%, 50%, and 75% of the runtime tokens are suppressed, respectively. In contrast, AdaGuard's selective-fusion prediction remains highly stable, changing from 0.7589 to 0.7564 ± 0.0008, 0.7555 ± 0.0026, and 0.7515 ± 0.0035. Even under 75% token suppression, the dynamic expert loses 14 percentage points in F1-score, whereas AdaGuard decreases by only 0.74 percentage points. This limited degradation indicates that the confidence-based selective-fusion mechanism effectively prevents incomplete runtime evidence from dominating the final decision.

This controlled suppression experiment is intended to approximate incomplete runtime traces that may arise from limited sandbox execution coverage or behavior suppression. This experiment does not constitute an exhaustive evaluation against fully adaptive sandbox-aware malware employing dedicated anti-analysis or environment-fingerprinting strategies.

Overall, the results answer RQ3 from complementary perspectives. The five-seed evaluation demonstrates that temperature smoothing improves the probabilistic reliability of dynamic predictions, while the static-confidence score consistently concentrates prediction errors in the uncertain subset. Selective invocation avoids unnecessary dynamic-expert calls without changing the hard-label performance of Always-Fusion, and the controlled degradation experiment further demonstrates that AdaGuard's late-fusion mechanism mitigates the impact of incomplete runtime evidence by maintaining stable final predictions even when a large portion of runtime tokens is suppressed.

Table 11
Runtime-evidence degradation analysis under the 10% target-year label budget. The perturbed conditions are reported as mean ± standard deviation over three perturbation seeds, while the 0% suppression condition denotes the clean reference without random perturbation.

Runtime-token suppression	Dynamic expert F1	AdaGuard F1	Dynamic drop (pp)	AdaGuard drop (pp)
0%	0.7069	0.7589	0.00	0.00
25%	0.6897 ± 0.0064	0.7564 ± 0.0008	1.72	0.25
50%	0.6413 ± 0.0127	0.7555 ± 0.0026	6.56	0.34
75%	0.5669 ± 0.0053	0.7515 ± 0.0035	14.00	0.74

Table 12
Description of ablation variants.

Variant	Removed component	Evaluation focus
Full AdaGuard	None	Complete model with all proposed components.
w/o Semantic edges	TransE-based API-cluster semantic edges	API-cluster semantic relations.
w/o Feature masking	Learnable feature masking module	Suppression of redundant and drift-sensitive static features.
w/o Dynamic channel	Dynamic runtime-evidence channel	Contribution of runtime-token evidence.
w/o EPA	Elastic malicious prototype alignment	Historical malware representation anchoring.
w/o KD	Knowledge distillation from previous static expert	Consistency with previous-stage decisions.
w/o GRL	Gradient-reversal-based domain alignment	Reduction of year-specific representation bias.

Table 13
Ablation results under chronological evaluation with the 10% target-year label budget.

Variant	Target-year F1 (%)				Avg._{tgt}
	2014	2015	2016	2017	F1-score (%)
Full AdaGuard	76.08	71.81	75.59	82.83	76.58
w/o Semantic edges	69.81	70.97	77.70	82.23	75.18
w/o Feature masking	72.50	70.66	75.51	82.01	75.17
w/o Dynamic channel	75.00	69.21	75.08	82.19	75.37
w/o EPA	73.85	71.54	76.94	80.47	75.70
w/o KD	74.26	71.26	77.43	82.11	76.27
w/o GRL	71.00	73.05	74.93	80.41	74.85

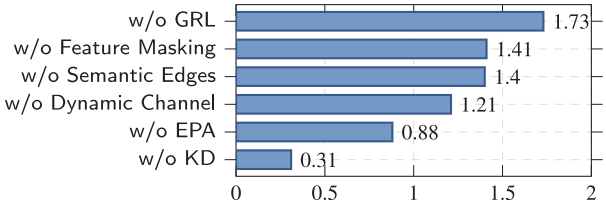


Fig. 6. Average target-year F1-score drop caused by removing each component under the 10% target-year label budget. Larger values indicate a greater contribution of the removed component to AdaGuard's temporal robustness.

4.2.4. RQ4: Ablation study on key components

To answer RQ4, we conduct ablation experiments to examine the contribution of each key component in AdaGuard. All ablation variants are evaluated under the same chronological protocol with the 10% target-year label budget, so that the performance differences can be attributed to the removed components. Table 12 summarizes the removed component and evaluation focus of each variant. Table 13 reports the year-wise F1-scores and the average target-year F1-score, while Fig. 6 visualizes the average F1-score degradation caused by removing each component.

As shown in Table 13, removing any key component leads to a decrease in the average target-year F1-score. Full AdaGuard achieves an Avg._{tgt} F1-score of 0.7658, while all ablated variants obtain lower average performance. Among them, removing GRL causes the largest degradation, reducing the Avg._{tgt} F1-score to 0.7485. Removing the feature masking module and semantic edges also leads to clear performance drops, with Avg._{tgt} F1-scores of 0.7517 and 0.7518, respectively. As further summarized in Fig. 6, the corresponding average F1-score drops are 1.73, 1.41, and 1.40 percentage points. These results indicate that AdaGuard's temporal robustness is not driven by a single component, but by the joint contribution of representation alignment, feature selection, and semantic graph modeling.

The largest degradation is observed when GRL is removed, which leads to an average F1-score drop of 1.73 percentage points. This indicates that year-specific representation bias is a major factor affecting temporal adaptation. By using gradient-reversal-based domain alignment, AdaGuard encourages the static encoder to learn representations that are less sensitive to the difference between historical and target-year samples. Removing feature masking also causes a clear drop of 1.41 percentage points, showing that the static feature space contains redundant or drift-sensitive dimensions. The learnable feature masking module helps suppress such unstable signals before graph aggregation. Similarly, removing semantic edges results in a 1.40-point decrease, which is almost comparable to the effect of removing feature masking. This confirms that TransE-based API-cluster semantic edges are important for capturing relations among API clusters and improving the robustness of the static graph channel under API and malware evolution. Removing the dynamic channel decreases the Avg._{tgt} F1-score from 0.7658 to 0.7537, corresponding to a 1.21-point drop. This demonstrates that runtime-token evidence provides useful complementary information beyond the static graph channel. Although the static graph channel serves as the full-coverage backbone of AdaGuard, the dynamic channel helps improve detection when static structural evidence is insufficient or ambiguous. Removing EPA also reduces the average F1-score by 0.88 points, indicating that historical malicious prototype anchoring helps stabilize the malware representation space during target-year adaptation. Removing KD causes the smallest degradation, with an average drop of 0.31 points. This result is reasonable because AdaGuard updates the static expert using the full training memory H_t, rather than adapting only on newly labeled target-year samples. Therefore, KD mainly serves as an auxiliary constraint that smooths stage-wise updating and helps maintain prediction consistency with the previous static expert.

Overall, the ablation results answer RQ4 by confirming that each key component contributes to AdaGuard's temporal robustness. GRL, feature masking, and semantic edges provide the largest gains, indicating the importance of reducing year-specific representation bias, suppressing drift-sensitive static features, and modeling API-cluster semantic relations. The dynamic channel and EPA further improve

Table 14
Graph-size comparison between the raw mapped-API and compact API-cluster representations.

Representation	APK nodes	Semantic nodes	Total nodes	APK–semantic edges
Raw mapped-API	22,788	23,001	45,789	49,366,075
AdaGuard compact	22,788	2000	24,788	16,876,967
Reduction	–	**91.30%**	**45.86%**	**65.81%**

Table 15
Graph-construction scalability with increasing APK population.

Scale (%)	No. of APKs	Construction time (s)	Additional peak RSS (GB)
20	4558	10.67 ± 0.69	1.05 ± 0.15
40	9115	22.06 ± 1.09	2.11 ± 0.30
60	13,673	33.83 ± 2.08	3.29 ± 0.36
80	18,230	52.02 ± 3.29	4.51 ± 0.36
100	22,788	63.45 ± 2.95	5.71 ± 0.36

robustness by incorporating runtime evidence and stabilizing the malware representation space, while KD provides an auxiliary constraint for smoother stage-wise updating. These results show that AdaGuard's performance is achieved through the joint effect of static semantic graph modeling, dynamic runtime evidence, and memory-anchored temporal adaptation.

4.2.5. RQ5: Computational cost and scalability of the semantic graph

To answer RQ5, we evaluate the compact APK–API-cluster semantic graph from three complementary aspects: representation compactness, graph-construction scalability, and incremental graph-update efficiency. For the runtime and memory analysis, graph profiling starts from the already extracted static features and therefore excludes APK decompilation, Smali parsing, static-feature extraction, model training, and dynamic sandbox execution. All runtime and memory measurements are conducted on the same Linux server and repeated three times, with the mean and standard deviation reported. Memory overhead is measured using the additional peak resident set size (RSS) incurred during graph construction or updating. The raw mapped-API versus compact API-cluster comparison is used only to quantify graph size.

Table 14 first quantifies the compactness introduced by the API-cluster abstraction. To ensure a consistent comparison, the raw graph considers the individual API nodes covered by the same APIGraph mapping used to construct the semantic clusters, rather than application-specific or unmapped methods that do not participate in the clustering process. Under this common semantic space, the raw representation contains 23,001 API nodes, whereas AdaGuard represents them using only $N_c = 2000$ API-cluster nodes, corresponding to a 91.30% reduction in semantic nodes. With the same 22,788 APK nodes retained in both representations, the total number of graph nodes decreases from 45,789 to 24,788, a reduction of 45.86%. In addition, API clustering aggregates repeated APK–API relations into APK–cluster relations, reducing the number of APK–semantic edges from 49.37 million to 16.88 million, or by 65.81%. These results provide direct quantitative evidence for the compact graph design described in Section 3.3.

The cluster-to-cluster semantic backbone incurs a one-time construction overhead. With $N_c = 2000$, the construction considers all 1,999,000 pairwise combinations of API clusters and retains 5253 semantic links according to the similarity criterion defined in Section 3.3.3. The centroid construction and pairwise similarity computation require 12.84 s in total, with an additional peak RSS of 11.30 MB. Since the API-cluster mapping and the cluster-to-cluster semantic backbone remain fixed during chronological updating, this cost is incurred only once and does not need to be repeated when new target-year APKs are incorporated.

We next examine how the APK-dependent graph construction cost changes as the application population increases. Specifically, we construct nested graph instances containing 20%, 40%, 60%, 80%, and 100% of the complete 22,788-APK dataset. The same fixed API-cluster semantic backbone is reused at all scales so that the measurements isolate the cost of constructing APK nodes and their associated APK–cluster edges. Table 15 reports the resulting construction time and additional peak RSS.

As shown in Table 15, the construction time of APK nodes and APK–cluster edges increases from 10.67 ± 0.69 s for 4558 APKs to 63.45 ± 2.95 s for the complete 22,788-APK dataset. A linear fit between the APK population and construction time yields $R^2 = 0.993$, indicating an approximately linear growth trend over the evaluated range. This observation is consistent with the APK-to-cluster construction complexity analyzed in Section 3.3.4: with the number of API clusters fixed at $N_c = 2000$, the construction cost grows linearly with the number of APK nodes. The additional peak RSS increases from 1.05 ± 0.15 GB to 5.71 ± 0.36 GB over the same range. Overall, both construction time and memory consumption exhibit predictable growth as the APK population increases within the evaluated dataset scale.

We further evaluate the cost of maintaining the semantic graph during chronological adaptation. Since the API-cluster mapping and the cluster-to-cluster semantic backbone remain fixed, a target-year update can reuse the existing graph and attach only the newly available APKs in L_t^{tr}. We therefore compare this incremental strategy with full reconstruction of all historical APK nodes and their associated APK–cluster edges corresponding to H_t. Both strategies reuse the same fixed semantic backbone, so the comparison isolates the computational cost of maintaining the APK-dependent part of the graph.

As shown in Table 16, only 279–311 newly labeled target-year APKs need to be attached at each chronological update under the 10% label budget, whereas the accumulated training memory H_t increases from 3935 to 4834 APKs. Consequently, incremental updating requires only 0.599–0.835 s, compared with 5.405–7.744 s for full APK-dependent graph reconstruction. This corresponds to a time reduction of 88.34%–89.22%, with an average reduction of 88.81%. The memory overhead is also substantially reduced: incremental updating incurs 54.96–72.64 MB of additional peak RSS, whereas full reconstruction requires 584.72–805.67 MB. On average, incremental updating reduces the additional peak-memory overhead by 90.65%.

Overall, the results answer RQ5 from both graph representation and graph maintenance perspectives. The API-cluster abstraction substantially reduces the semantic graph size, while the APK-dependent construction cost exhibits approximately linear growth with the number of APKs over the evaluated range. Moreover, reusing the fixed semantic backbone and attaching only newly available target-year APKs substantially reduces both update time and memory overhead compared with reconstructing the accumulated APK-dependent graph component. These results demonstrate that AdaGuard's compact semantic graph has predictable construction costs and supports efficient incremental maintenance under the chronological deployment setting considered in this study.

4.2.6. Temporal generalization evaluation on recent Android applications

To further evaluate the temporal generalization capability of AdaGuard under recent Android evolution, we conduct additional experiments on a newly collected Android malware dataset from the AndroZoo repository, covering applications released between 2022 and 2025. Although the original experiments were conducted on AndroZoo samples from 2013 to 2017, the Android ecosystem has continuously evolved in recent years, accompanied by changes in application behaviors, API usage patterns, and malware characteristics. Therefore, this additional evaluation investigates whether AdaGuard can maintain

Table 16
Comparison of full APK-dependent graph reconstruction and incremental graph updating under the 10% target-year label budget.

Year	$\|L_t^{tr}\|$	$\|H_t\|$	Incremental time (s)	Full reconstruction time (s)	Time reduction (%)	Incremental RSS (MB)	Full reconstruction RSS (MB)
2014	279	3935	0.599 ± 0.106	5.405 ± 0.556	88.92	54.96	584.72
2015	288	4223	0.682 ± 0.053	6.077 ± 0.370	88.77	61.34	651.08
2016	311	4534	0.815 ± 0.068	6.989 ± 0.628	88.34	72.64	729.78
2017	300	4834	0.835 ± 0.068	7.744 ± 0.453	89.22	69.36	805.67

Table 17
Overview of the temporal dataset (2022–2025).

Year	2022	2023	2024	2025	Total
Malware	309	297	334	291	1231
Benign	597	749	670	559	2575
Total	906	1046	1004	850	3806

its detection advantage against the strongest updated baseline on recent Android applications collected from a previously unseen temporal period.

The newly collected dataset contains 3806 Android applications released between 2022 and 2025, including 1231 malware samples and 2575 benign samples. The detailed distribution of malware and benign applications across different years is summarized in Table 17. Following the temporal evaluation protocol of AdaGuard, applications collected in 2022 are used as the source domain, while applications from 2023 to 2025 are regarded as sequential target domains for incremental adaptation. Since the 2022 source-domain samples are fixed and the labeling budget is only applied to the target domains, the results on the 2022 source domain remain unchanged across different labeling budgets and random seeds. Among all baseline methods evaluated in the original experiments, Drebin-SVM-Upd achieved the strongest performance; therefore, it is selected as the representative baseline for focused comparison under the same temporal adaptation setting. For the target domains, we randomly select labeled samples with budgets of 5%, 10%, and 15%, respectively, and repeat each experiment with five independent random seeds. The results are reported as mean ± standard deviation of F1-score to evaluate both detection effectiveness and robustness against random sample selection.

To further clarify the temporal relationship between the application years and the VirusTotal-based metadata, Table 18 summarizes the APK date ranges and the recorded VirusTotal scan-date ranges for both the original 2013–2017 dataset and the recent 2022–2025 evaluation dataset. The application year is determined by the APK `dex_date`, whereas `vt_scan_date` is a separate metadata field and may fall in the same year or in a later year. The VirusTotal scan date is not used to define the chronological partition. As shown in Table 18, the recorded VirusTotal scan dates can substantially postdate the corresponding application years.

Table 19 presents the F1-score comparison between AdaGuard and Drebin-SVM-Upd on recent Android applications. Overall, AdaGuard achieves higher average target-year F1-scores than Drebin-SVM-Upd under all labeling budgets, demonstrating that AdaGuard maintains competitive detection performance under recent Android ecosystem evolution. Specifically, AdaGuard achieves average target-year F1-scores of 0.7821, 0.7930, and 0.7964 under the 5%, 10%, and 15% labeling budgets, respectively, compared with 0.7686, 0.7745, and 0.7793 achieved by Drebin-SVM-Upd. These results correspond to improvements of 1.35, 1.85, and 1.71 percentage points, respectively. The results indicate that AdaGuard can maintain more reliable detection performance when only limited labeled samples are available. Furthermore, the advantage of AdaGuard is more evident on the latest 2025 applications, which represent the most recent temporal period in this additional evaluation. For example, under the 10% labeling budget, AdaGuard achieves an F1-score of 0.6173, compared with 0.5699 achieved by Drebin-SVM-Upd, resulting in an improvement of 4.74 percentage points. Although both methods experience performance degradation as Android applications evolve over time, AdaGuard maintains competitive performance under recent temporal evolution by exploiting adaptive evidence and incremental information.

It is worth noting that the performance advantage of AdaGuard is not strictly consistent across all target years and labeling budgets. For example, Drebin-SVM-Upd slightly outperforms AdaGuard on the 2024 subset under the 10% labeling budget. This fluctuation may be related to random sample selection and the varying difficulty of different temporal domains. In some temporal periods where static features remain sufficiently discriminative, the additional adaptive evidence may provide limited marginal benefits. Moreover, increasing the labeling budget does not always guarantee higher performance, since additional labeled samples may have different distributions or may include more challenging instances. Nevertheless, the overall improvements across different budgets and the stronger performance on the latest 2025 applications demonstrate the effectiveness of AdaGuard under recent temporal evolution of Android applications.

5. Conclusion

This paper presented AdaGuard, a temporal-leakage-free framework for Android malware detection under chronological concept drift. AdaGuard combines a temporally isolated APK–API-cluster semantic graph, uncertainty-triggered runtime evidence, and memory-anchored temporal adaptation. This design enables the detector to exploit complementary static and dynamic information while updating year by year with limited target-year labels.

Experiments on Android applications collected from 2013 to 2017 show that AdaGuard consistently improves average target-year Coverage-aware F1 under 5%, 10%, and 15% labeling budgets. In particular, the multi-seed evaluation further shows that AdaGuard with only 5% labeled target-year data achieves an average target-year F1-score of 0.7218, closely matching the 0.7224 obtained by the strongest updated baseline using a 15% labeling budget, demonstrating strong label efficiency and competitive temporal adaptation performance. The ablation results further confirm the contributions of the semantic graph design, learnable feature masking, uncertainty-triggered dynamic evidence, and memory-anchored adaptation.

The present evaluation mainly characterizes chronological concept drift as reflected in observable static and dynamic evidence, while different malware-evolution mechanisms may affect these evidence channels differently. Although AdaGuard combines API-cluster indicators with permission features, opcode statistics, and complementary runtime evidence, more sophisticated transformations such as control-flow obfuscation, reflection, dynamic code loading, packing, or encrypted payloads may reduce the completeness of statically observable behaviors, while anti-analysis or sandbox-evasion techniques may suppress runtime evidence. The controlled runtime-evidence degradation experiment provides a controlled analysis of incomplete runtime evidence, which represents one possible consequence of runtime suppression mechanisms, but the current study does not constitute a transformation-specific robustness benchmark against dedicated obfuscation, packing/encryption, or fully adaptive anti-analysis attacks.

Future work will extend AdaGuard in three directions. First, more informative target-year sample-selection strategies will be investigated

Table 18
APK date ranges and VirusTotal scan-date distributions for the original and recent temporal evaluation datasets.

Application year	Final APKs	`dex_date` range	VirusTotal `vt_scan_date` range	Same-year scans	Later-year scans
2013	4352	2013-01-01–2013-12-31	2013-01-06–2024-01-25	1724 (39.61%)	2628 (60.39%)
2014	4366	2014-01-01–2014-12-31	2014-01-06–2024-01-24	3056 (70.00%)	1310 (30.00%)
2015	4500	2015-01-01–2015-12-31	2015-01-11–2024-01-25	811 (18.02%)	3689 (81.98%)
2016	4872	2016-01-01–2016-12-31	2016-01-04–2024-04-09	3484 (71.51%)	1388 (28.49%)
2017	4698	2017-01-01–2017-12-31	2017-01-03–2025-01-27	1555 (33.10%)	3143 (66.90%)
2022	906	2022-01-01–2022-12-31	2022-02-08–2025-11-12	785 (86.64%)	121 (13.36%)
2023	1046	2023-01-01–2023-12-29	2023-02-01–2025-11-13	835 (79.83%)	211 (20.17%)
2024	1004	2024-01-02–2024-12-31	2024-02-14–2025-11-13	696 (69.32%)	308 (30.68%)
2025	850	2025-01-01–2025-10-10	2025-02-04–2025-11-12	850 (100.00%)	0 (0.00%)

Note: The application year is determined by the APK `dex_date`. The recorded VirusTotal scan date is a separate metadata field and is not used to define the temporal partition.

Table 19
F1 stability of AdaGuard and Drebin-SVM-Upd under different random selections of labeled samples on recent Android applications (2022–2025). Results are reported as mean ± standard deviation over five random seeds.

Label budget	2022		2023		2024		2025		$Avg._{tgt}$ F1-score	
	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd	AdaGuard	Drebin-SVM-Upd
5%	**0.9667 ± 0.0000**	0.9194 ± 0.0000	**0.9134 ± 0.0060**	0.9035 ± 0.0071	**0.8537 ± 0.0250**	0.8451 ± 0.0117	**0.5791 ± 0.0470**	0.5571 ± 0.0796	**0.7821 ± 0.0166**	0.7686 ± 0.0245
10%	**0.9667 ± 0.0000**	0.9194 ± 0.0000	**0.9168 ± 0.0076**	0.9002 ± 0.0055	0.8449 ± 0.0268	**0.8534 ± 0.0184**	**0.6173 ± 0.0434**	0.5699 ± 0.0659	**0.7930 ± 0.0190**	0.7745 ± 0.0161
15%	**0.9667 ± 0.0000**	0.9194 ± 0.0000	**0.9168 ± 0.0073**	0.8951 ± 0.0143	**0.8385 ± 0.0126**	0.8264 ± 0.0272	**0.6340 ± 0.0472**	0.6164 ± 0.0497	**0.7964 ± 0.0145**	0.7793 ± 0.0114

to replace random annotation, for example by jointly considering predictive uncertainty, sample diversity, and temporal novelty under the same labeling budget. Second, the dynamic channel will be strengthened through more reliable runtime-trace collection, multi-run behavior aggregation, and cost-aware evidence triggering, with particular attention to applications for which valid dynamic traces are incomplete or unavailable. Third, AdaGuard will be evaluated in longer and more diverse deployment settings involving additional years, malware families, application markets, and previously unseen threats, together with transformation-specific robustness analyses covering obfuscation, packing/encryption, and anti-analysis behaviors. This direction will further investigate finer-grained continual adaptation and adaptive memory management to improve robustness and stability under long-term deployment.

CRediT authorship contribution statement

Linfeng Yang: Writing – review & editing, Funding acquisition, Conceptualization. **Zhi Xie:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Conceptualization. **Zhiding Wu:** Writing – review & editing, Validation. **Yuan Meng:** Writing – review & editing, Validation. **Zhenrong Yang:** Writing – review & editing, Validation.

Funding

This work was supported by the Key Research and Development Program of Guangxi (GuiKe-AB23075199) and the Natural Science Foundation of Guangxi (2026GXNSFHA00640030).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The code and data associated with this study can be found at: https://github.com/linfengYang/AdaGuard.

References

Allix, K., Bissyandé, T.F., Klein, J., Le Traon, Y., 2016. AndroZoo: Collecting millions of Android apps for the research community. In: Proceedings of the 13th International Conference on Mining Software Repositories. Association for Computing Machinery, pp. 468–471. http://dx.doi.org/10.1145/2901739.2903508.

Arp, D., Spreitzenbarth, M., Hübner, M., Gascon, H., Rieck, K., 2014. DREBIN: Effective and explainable detection of Android malware in your pocket. In: Proceedings of the 2014 Network and Distributed System Security Symposium. NDSS, Internet Society, http://dx.doi.org/10.14722/ndss.2014.23247.

Barbero, F., Pendlebury, F., Pierazzi, F., Cavallaro, L., 2022. Transcending TRANSCEND: Revisiting malware classification in the presence of concept drift. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 805–823. http://dx.doi.org/10.1109/SP46214.2022.9833659.

Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., Yakhnenko, O., 2013. Translating embeddings for modeling multi-relational data. In: Advances in Neural Information Processing Systems. Vol. 26, pp. 2787–2795.

Ceschin, F., Botacin, M., Gomes, H.M., Pinagé, F., Oliveira, L.S., Grégio, A., 2023. Fast & furious: On the modelling of malware detection as an evolving data stream. Expert Syst. Appl. 212, 118590. http://dx.doi.org/10.1016/j.eswa.2022.118590.

Chen, Y., Ding, Z., Wagner, D., 2023. Continuous learning for Android malware detection. In: 32nd USENIX Security Symposium. USENIX Security 23, USENIX Association, pp. 1127–1144, URL: https://www.usenix.org/conference/usenixsecurity23/presentation/chen-yizheng.

Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., et al., 2016. Domain-adversarial training of neural networks. J. Mach. Learn. Res. 17 (59), 1–35.

Goodfellow, I., Bengio, Y., Courville, A., 2016. Deep Learning. MIT Press, Cambridge, MA.

Guerra-Manzanares, A., 2024. Machine learning for android malware detection: Mission accomplished? a comprehensive review of open challenges and future perspectives. Comput. Secur. 138, 103654. http://dx.doi.org/10.1016/j.cose.2023.103654.

Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q., 2017. On calibration of modern neural networks. In: Proceedings of the 34th International Conference on Machine Learning. In: Proceedings of Machine Learning Research, Vol. 70, PMLR, pp. 1321–1330, URL: https://proceedings.mlr.press/v70/guo17a.html.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems. Vol. 30, pp. 1024–1034.

Haque, M.A., Hossain, I., Kamol, M.M., Alam, M.J., Amalapuram, S.K., Talukder, S., et al., 2026. LAMDA: A longitudinal Android malware benchmark for concept drift analysis. In: The 14th International Conference on Learning Representations. ICLR, URL: https://openreview.net/forum?id=1FnCrZtBNQ.

He, Y., Lei, J., Qin, Z., Ren, K., Chen, C., 2025. Combating concept drift with explanatory detection and adaptation for Android malware classification. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, pp. 978–992. http://dx.doi.org/10.1145/3719027.3744792.

He, Y., Liu, Y., Wu, L., Yang, Z., Ren, K., Qin, Z., 2023. MsDroid: Identifying malicious snippets for Android malware detection. IEEE Trans. Dependable Secur. Comput. 20 (3), 2025–2039. http://dx.doi.org/10.1109/TDSC.2022.3168285.

Hei, Y., Yang, R., Peng, H., Wang, L., Xu, X., Liu, J., et al., 2024. HAWK: Rapid Android malware detection through heterogeneous graph attention networks. IEEE Trans. Neural Netw. Learn. Syst. 35 (4), 4703–4717. http://dx.doi.org/10.1109/TNNLS.2021.3105617.

Hinton, G., Vinyals, O., Dean, J., 2015. Distilling the knowledge in a neural network. http://dx.doi.org/10.48550/arXiv.1503.02531, arXiv preprint arXiv:1503.02531.

Huang, L., Xue, J., Wang, Y., Chen, J., Lei, T., 2024. Strengthening LLM ecosystem security: Preventing mobile malware from manipulating LLM-based applications. Inf. Sci. 681, 120923. http://dx.doi.org/10.1016/j.ins.2024.120923.

Liu, Z., Wang, R., Peng, B., Qiu, L., Gan, Q., Wang, C., et al., 2025. LDCDroid: Learning data drift characteristics for handling the model aging problem in Android malware detection. Comput. Secur. 150, 104294. http://dx.doi.org/10.1016/j.cose.2024.104294.

McFadden, S., Foley, M., D'Onghia, M., Hicks, C., Mavroudis, V., Paoletti, N., et al., 2026. DRMD: Deep reinforcement learning for malware detection under concept drift. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 40, AAAI Press, pp. 854–862. http://dx.doi.org/10.1609/aaai.v40i2.37053.

Muzaffar, A., Hassen, H.R., Lones, M.A., Zantout, H., 2022. An in-depth review of machine learning based Android malware detection. Comput. Secur. 121, 102833. http://dx.doi.org/10.1016/j.cose.2022.102833.

Oliveira, A.S.d., Sassi, R.J., 2020. Chimera: An Android malware detection method based on multimodal deep learning and hybrid analysis. http://dx.doi.org/10.36227/techrxiv.13359767.v1, TechRxiv preprint.

Onwuzurike, L., Mariconti, E., Andriotis, P., De Cristofaro, E., Ross, G., Stringhini, G., 2019. MaMaDroid: Detecting Android malware by building Markov chains of behavioral models (extended version). ACM Trans. Priv. Secur. 22 (2), 14:1–14:34. http://dx.doi.org/10.1145/3313391.

Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L., 2019. TESSERACT: Eliminating experimental bias in malware classification across space and time. In: 28th USENIX Security Symposium (USENIX Security 19). USENIX Association, pp. 729–746, URL: https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury.

Sabbah, A., Jarrar, R., Zein, S., Mohaisen, D., 2025. Empirical evaluation of concept drift in ML-based Android malware detection. http://dx.doi.org/10.48550/arXiv.2507.22772, arXiv preprint arXiv:2507.22772.

Salton, G., Wong, A., Yang, C.-S., 1975. A vector space model for automatic indexing. Commun. ACM 18 (11), 613–620. http://dx.doi.org/10.1145/361219.361220.

Shen, L., Fang, M., Xu, J., 2024. GHGDroid: Global heterogeneous graph-based Android malware detection. Comput. Secur. 141, 103846. http://dx.doi.org/10.1016/j.cose.2024.103846.

Sparck Jones, K., 1972. A statistical interpretation of term specificity and its application in retrieval. J. Doc. 28 (1), 11–21. http://dx.doi.org/10.1108/eb026526.

Sun, T., Daoudi, N., Pian, W., Kim, K., Allix, K., Bissyandé, T.F., et al., 2025. Temporal-incremental learning for Android malware detection. ACM Trans. Softw. Eng. Methodol. 34 (4), 101:1–101:30. http://dx.doi.org/10.1145/3702990.

Taher, F., AlFandi, O., Al-kfairy, M., Al Hamadi, H., Alrabaee, S., 2023. DroidDetectMW: A hybrid intelligent model for Android malware detection. Appl. Sci. 13 (13), 7720. http://dx.doi.org/10.3390/app13137720.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks. In: International Conference on Learning Representations. ICLR, URL: https://openreview.net/forum?id=rJXMpikCZ.

Wang, H., Zhang, W., He, H., 2022. You are what the permissions told me! Android malware detection based on hybrid tactics. J. Inf. Secur. Appl. 66, 103159. http://dx.doi.org/10.1016/j.jisa.2022.103159.

Wu, Y., Shi, J., Wang, P., Zeng, D., Sun, C., 2023. DeepCatra: Learning flow- and graph-based behaviours for Android malware detection. IET Inf. Secur. 17 (1), 118–130. http://dx.doi.org/10.1049/ise2.12082.

Wu, M., Zheng, X., Zhang, Q., Shen, X., Luo, X., Zhu, X., et al., 2024. Graph learning under distribution shifts: A comprehensive survey on domain adaptation, out-of-distribution, and continual learning. http://dx.doi.org/10.48550/arXiv.2402.16374, arXiv preprint arXiv:2402.16374.

Xu, J., Li, Y., Deng, R.H., Xu, K., 2022. SDAC: A slow-aging solution for Android malware detection using semantic distance based API clustering. IEEE Trans. Dependable Secur. Comput. 19 (2), 1149–1163. http://dx.doi.org/10.1109/TDSC.2020.3005088.

Yang, Y., Du, X., Yang, Z., Liu, X., 2021. Android malware detection based on structural features of the function call graph. Electronics 10 (2), 186. http://dx.doi.org/10.3390/electronics10020186.

Yang, J., Liang, H., Ren, H., Jia, D., Wang, X., 2025. SAC: Collaborative learning of structure and content features for Android malware detection framework. Neurocomputing 637, 130053. http://dx.doi.org/10.1016/j.neucom.2025.130053.

Yumlembam, R., Issac, B., Jacob, S.M., Yang, L., 2023. IoT-based Android malware detection using graph neural network with adversarial defense. IEEE Internet Things J. 10 (10), 8432–8444. http://dx.doi.org/10.1109/JIOT.2022.3188583.

Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R.R., Smola, A.J., 2017. Deep sets. In: Advances in Neural Information Processing Systems. Vol. 30, pp. 3391–3401.

Zhang, X., Zhang, Y., Zhong, M., Ding, D., Cao, Y., Zhang, Y., et al., 2020. Enhancing state-of-the-art classifiers with API semantics to detect evolved Android malware. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, pp. 757–770. http://dx.doi.org/10.1145/3372297.3417291.